

**DEEPFIT: AI-POWERED PLATFORM FOR SPORTS
TALENT ASSESSMENT**

*A Report
Submitted in partial fulfilment of the
Requirements for the completion of*

**THEME BASED PROJECT
BACHELOR OF ENGINEERING
IN
INFORMATION TECHNOLOGY**

By

MALLARAPU ARUN CHAND - 1602-23-737-010

Under the guidance of

**DR. S.K PRASHANTH
(ASSOCIATE PROFESSOR)**



**Department of Information Technology
Vasavi College of Engineering (Autonomous)
ACCREDITED BY NAAC WITH 'A++' GRADE.
(Affiliated to Osmania University and Approved by AICTE)**

Ibrahim Bagh, Hyderabad-31

2026

**Vasavi College of Engineering (Autonomous)
ACCREDITED BY NAAC WITH 'A++' GRADE**

(Affiliated to Osmania University and Approved by AICTE)

Ibrahim Bagh, Hyderabad-31

Department of Information Technology



DECLARATION BY CANDIDATES

MALLARAPU ARUN CHAND bearing hall ticket number, **1602-23-737-010** hereby declare that the project report entitled "**DEEPFIT: AI-POWERED PLATFORM FOR SPORTS TALENT ASSESSMENT**" under the guidance of **DR. S.K PRASHANTH, ASSOCIATE PROFESSOR**, Department of Information Technology, Vasavi College of Engineering, Hyderabad, is submitted in partial fulfilment of the requirement for the completion of Theme-based project, VI semester, Bachelor of Engineering in Information Technology.

This is a record of bonafide work carried out by us and the results embodied in this project report have not been submitted to any other institutes.

MALLARAPU ARUN CHAND - 1602-23-737-010

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University and Approved by AICTE)

Ibrahim Bagh, Hyderabad-31

Department of Information Technology



BONAFIDE CERTIFICATE

This is to certify that the project entitled "**DEEPFIT: AI-POWERED PLATFORM FOR SPORTS TALENT ASSESSMENT**" being submitted by **MALLARAPU ARUN CHAND** bearing **1602-23-737-010**, in partial fulfilment of the requirements for the completion of Theme-based project of Bachelor of Engineering in Information Technology is a record of bonafide work carried out by them under my guidance.

Dr. S.K Prashanth,
Associate Professor, IT

External Examiner

Dr. K. Ram MohanRao,
Professor, HOD IT

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the project would not have been possible without the kind support and help of many individuals. We would like to extend our sincere thanks to all of them.

It is with immense pleasure that we would like to take the opportunity to express our humble gratitude to **DR. S.K PRASHANTH, ASSOCIATE PROFESSOR, Information Technology** under whom we executed this project. Her constant guidance and willingness to share vast knowledge made us understand this project and its manifestations in great depths and helped us to complete the assigned tasks.

We are very much thankful to **Dr. K. Ram Mohan Rao, Professor and HOD, Information Technology**, for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **Dr. S. V. Ramana, Principal of Vasavi College of Engineering** for giving the required information in doing this project work. Not to forget, we thank all other faculty and non-teaching staff, and our friends who had directly or indirectly helped and supported us in completing our project in time.

We also express our sincere thanks to the Management for providing excellent facilities. Finally, we wish to convey our gratitude to our families who fostered all the facilities that we needed.

ABSTRACT

DeepFit AI is an AI-powered sports talent assessment platform that improves traditional fitness evaluations, which often depend on manual supervision and subjective judgment, leading to errors and bias, especially in rural areas. It supports modules like **Height and Weight** (BMI-based assessment), **Sit-ups, Vertical Jump, Sit-and-Reach Flexibility Test, and Dumbbell Curl**, delivering instant analytics, graphical feedback, and secure database management for transparent and unbiased evaluation. Using a single camera, the system leverages computer vision, machine learning, and Pose Detection to analyze body movements in **real time**, accurately measuring repetitions, jump height, flexibility, and strength. By enabling accessible, scalable, and data-driven assessments, DeepFit AI ensures **fair sports talent identification** for academies and large-scale talent discovery programs.

Furthermore, the project is designed to be inclusive by supporting the assessment of differently-abled athletes, including blind and deaf individuals. DeepFit AI incorporates adaptive evaluation techniques to analyse both physical and cognitive aspects such as coordination, reaction patterns, and mental focus through visual cues and movement-based analysis. The system ensures that **disabled athletes** are assessed fairly without reliance on verbal instructions, making the evaluation process accessible, **unbiased**, and standardized. This inclusive approach promotes equal opportunities in sports talent identification by recognizing the capabilities of all individuals, regardless of physical or sensory limitations. In addition, DeepFit AI extends its functionality through a dedicated **Mobile App** developed using **Android Studio**, ensuring portability and ease of access for users in diverse environments. The mobile app integrates secure tunneling via ngrok to connect with backend services in real time, enabling seamless data transmission, live assessment tracking, and instant feedback visualization. This allows coaches, athletes, and administrators to perform and monitor evaluations directly from mobile devices, making the system highly scalable and suitable for on-field usage, remote locations, and large-scale deployment scenarios.

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1 Overview	1
1.2 Problem Statement	1
1.3 Motivation of Theme & Title	2
2. LITERATURE SURVEY	4
3. EXISTING SYSTEM	5
4. PROPOSED SOLUTION	6
4.1 System Design	6
4.1.1 Architecture Diagram	7
4.1.2 Use-Case Diagram	9
4.1.2.1 Use-Case Descriptions	9
4.1.3 All UML Diagrams	12
4.2 Functional Modules	18
4.2.1 Screenshots	20
5. EXPERIMENTAL SETUP & IMPLEMENTATION	41
5.1 System Specifications	41
5.1.1 Hardware Requirements	41
5.1.2 Software Requirements	41
5.2 Datasets & Code Implementation	42
5.3 Methodology / Algorithm	60
6. RESULTS	62
7. CONCLUSION & FUTURE SCOPE	63
8. REFERENCES	64

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
CV	Computer Vision
API	Application Programming Interface
HTTPS	Hyper Text Transfer Protocol Secure
DB	Database
DL	Deep Learning
CNN	Convolutional Neural Network
JSON	JavaScript Object Notation
FPS	Frames Per Second
TTS	Text-to-Speech
STT	Speech-to-Text
YOLO	You Only Look Once
SAI	Sports Authority of India

1. INTRODUCTION

1.1 Overview

Physical fitness assessments play an important role in identifying sports talent and monitoring athlete performance. However, most current evaluation methods depend on **manual observation** and **physical measuring** tools such as stopwatches, measuring tapes, and human counting. These approaches are time-consuming, prone to **human error**, and often require trained officials and **physical infrastructure**.

In many regions, especially rural and under-resourced areas, athletes do not have access to proper testing facilities or sports academies. This limits equal participation and makes large-scale talent identification difficult. To address these challenges, there is a need for a digital system that can perform accurate fitness assessments using modern technologies such as computer vision, artificial intelligence, and real-time motion analysis [7]. Such a system can significantly reduce dependency on physical infrastructure and trained personnel. It also enables scalable, cost-effective, and remote assessment, ensuring wider reach and inclusivity in talent identification programs.

In addition to general fitness assessment, DeepFit AI incorporates a dedicated module for disabled individuals, making the system inclusive and accessible. Blind users are provided with audio-based tests such as sound direction recognition and speech-based question answering, while deaf users are evaluated through visual-based exercises like Sit-ups and other. The integration of an AI chatbot further enhances the system by providing guidance, answering queries, and improving user interaction through voice and text-based communication. Overall, DeepFit AI offers a scalable, efficient, and intelligent solution that can be used across sports academies, schools, and remote locations using AI based pose estimation and exercise analysis techniques [8].

1.2 Problem Statement

Traditional sports fitness assessments rely on manual supervision, measuring instruments, and human judgment. These methods often lead to errors, inconsistent evaluation, higher operational costs, and limited accessibility, especially for athletes in rural or remote areas. There is a need for an automated, scalable, and unbiased system that can accurately evaluate athletes physical performance without requiring specialized equipment or centralized testing centers.

India has one of the largest populations in the world, yet it wins relatively few **Olympic medals** compared to its potential. One major reason is the lack of effective systems to identify the right athletes at an early stage and the presence of **biased** or **inconsistent evaluation** methods. Many talented athletes remain undiscovered due to limited access to proper assessment facilities. An AI-based platform can help remove bias and accurately evaluate athlete performance, enabling **fair talent identification** across the country.

Despite having a vast population, struggles to convert its potential into international sports success. One of the key reasons is the absence of a standardized and scalable system for talent identification. The presence of **evaluation bias** and inconsistent assessment methods further complicates the process. Therefore, there is a critical need for a system that can provide accurate, fair, and accessible fitness evaluation without relying on manual intervention or specialized infrastructure. Additionally, the platform incorporates features specifically designed for disabled athletes, aligning with the vision of the **Para Olympics**, by enabling the assessment of individuals with visual and hearing impairments. It focuses on evaluating both physical and cognitive abilities of **blind and deaf** athletes through adaptive, multiple choice questions audio based, and movement-based analysis, ensuring inclusive, unbiased, and standardized talent identification.

Current fitness assessment methods also lack real-time analytical feedback and proper digital monitoring, making it difficult to track athlete progress efficiently. Many assessment processes are time-consuming and depend heavily on trained personnel, reducing scalability in large-scale talent identification programs. An AI-powered automated system can provide faster evaluation, continuous monitoring, and data-driven insights to support better training and athlete development.

1.3 Motivation of Theme & Title

The primary motivation behind DeepFit AI is to revolutionize the traditional fitness assessment process by introducing an automated, intelligent, and unbiased system. The project aims to eliminate the limitations of manual evaluation and replace them with a data-driven approach that ensures consistency and accuracy. By removing **human bias** and **evaluation bias**, the system promotes fairness and transparency in athlete assessment.

Another important motivation is inclusivity. Many existing systems do not cater to individuals with disabilities, thereby excluding a significant portion of the population from participating in fitness assessments. DeepFit AI addresses this gap by introducing specialized modules for blind and deaf users, ensuring equal opportunities for all individuals regardless of their physical limitations.

The use of modern technologies such as computer vision and artificial intelligence also makes the system scalable and adaptable. It can be deployed in various environments, including schools, sports academies, and remote areas, using only a camera-enabled device. This reduces dependency on expensive equipment and trained personnel, making the system cost-effective and widely accessible. Overall, the motivation behind DeepFit AI is to create a smart, inclusive, and efficient platform for sports talent identification and performance evaluation.

The project is also motivated by the growing need for technology-driven solutions in the sports domain. With advancements in AI and machine learning, fitness evaluation can be made more precise, faster, and reliable compared to conventional methods. DeepFit AI encourages digital transformation in sports assessment by integrating intelligent monitoring, real-time feedback, and automated analysis. This helps athletes, trainers, and organizations make better decisions for performance improvement and talent development.

2. LITERATURE SURVEY

The fields of **computer vision** and **artificial intelligence** have experienced significant advancements in recent years, particularly in applications related to human activity recognition and **motion analysis**. Technologies such as **MediaPipe** enable real-time detection and tracking of human body landmarks with high precision, while **OpenCV** provides robust tools for **image and video processing**. Together, these technologies facilitate the development of efficient systems capable of analyzing human movements and extracting meaningful insights from visual data.

Machine learning frameworks such as **TensorFlow** and **PyTorch** have played a vital role in building intelligent models that can interpret complex human movements. These frameworks support the design of **neural networks** that learn from large datasets and generate accurate predictions. Research in this domain has led to the development of systems capable of counting repetitions, detecting incorrect posture, and providing real-time feedback, thereby enhancing performance analysis and training effectiveness.

In addition to motion analysis, advancements in **DeepFake detection** and **video forensics** have introduced techniques to verify the **authenticity** and integrity of visual data. This is particularly important in systems like DeepFit AI, where reliable input data is essential for accurate assessment outcomes. However, many existing solutions are primarily focused on **general fitness tracking** and lack critical features such as automated evaluation, bias reduction, and accessibility for differently-abled individuals. DeepFit AI addresses these limitations by integrating advanced technologies into a unified platform that ensures **accurate, fair, and inclusive fitness assessment** for all users, including **athletes with disabilities**.

3. EXISTING SYSTEM

Currently, sports fitness assessments are mostly conducted through manual testing supervised by coaches or officials using tools such as stopwatches, measuring tapes, and manual counting. These tests usually take place in sports academies or training camps, requiring athletes to travel to specific locations, which creates accessibility challenges for athletes from **rural** or underprivileged areas.

Manual evaluation can also lead to **human bias, measurement errors**, and inconsistent scoring. Although some digital fitness applications exist, they mainly focus on general fitness tracking and often require wearable devices or sensors. Therefore, existing solutions highlight the need for DeepFit AI, which can provide accurate, accessible, **low-cost** assessment and standardized fitness assessment.

Another limitation is the lack of accessibility, as athletes from rural or underprivileged areas often do not have access to proper training facilities or qualified coaches, restricting their ability to participate in assessments and limiting their chances of being identified as potential talent. Furthermore, existing digital solutions are primarily focused on personal fitness tracking and frequently require wearable devices or sensors, which may not be affordable or accessible to everyone.

In addition, the absence of centralized data management systems makes it difficult to **track athlete performance** over time or compare results across different regions, leading to fragmented evaluation processes and reduced transparency in talent selection. Many current systems also lack real-time feedback mechanisms, preventing athletes from instantly understanding their performance and areas for improvement. Moreover, existing solutions rarely address inclusivity, particularly for differently-abled athletes such as individuals with visual or hearing impairments, as there is limited support for audio based interaction, adaptive testing methods, or cognitive assessment.

4. PROPOSED SOLUTION

DeepFit AI is proposed as a comprehensive and intelligent solution to address the limitations of traditional sports fitness assessment systems. The platform utilizes a single-camera setup to capture video input and applies advanced computer vision and artificial intelligence techniques to analyse athlete performance in real time. By leveraging **pose detection algorithms**, the system accurately tracks body movements, eliminating the need for manual supervision or specialized equipment. This enables automated, consistent, and objective evaluation of athletes across different environments.

The system supports multiple fitness assessment modules, including **Sit-ups**, **Vertical Jump**, **Sit-and-Reach**, and **Dumbbell Curls**, each designed to evaluate specific physical attributes such as strength, flexibility, and endurance. Customized algorithms are used for each activity—for instance, **repetition counting** for strength-based exercises and joint angle analysis for flexibility measurement. Additionally, the platform integrates a **cheat detection mechanism**, incorporating techniques from **DeepFake detection** and video validation to ensure the authenticity and reliability of input data, thereby maintaining transparency and trust in the evaluation process.

A key feature of DeepFit AI is its strong focus on **inclusivity**. The platform is designed to support differently-abled athletes, including **blind** and **deaf** individuals, through adaptive and accessible evaluation methods. **Blind** users are guided using **audio-based instructions** and interaction mechanisms, while **deaf** users are assessed through visual cues and movement analysis. The system also includes an **AI-powered chatbot** to provide real-time assistance and guidance during assessments. Overall, DeepFit AI offers a **scalable**, **accurate**, and **unbiased** solution that promotes fair and inclusive sports talent identification.

The platform provides real-time performance analysis and detailed feedback to help athletes improve their fitness levels effectively. It evaluates posture accuracy, movement efficiency, and exercise performance using AI-based techniques. The system also maintains assessment records for progress tracking and offers a user-friendly interface for smooth interaction. By combining automation and intelligent analysis, it ensures accurate, fair, and efficient sports fitness evaluation

4.1 System Design

The DeepFit AI system follows a modular and scalable architecture designed to enable automated and unbiased fitness assessment using a set of models. The process begins with **user authentication**, where athletes securely register or log in to access the platform. Once validated, users proceed to the fitness evaluation stage, which includes **exercise selection**, **live video capture**, and dedicated assessment pathways for **blind** and **deaf** users. The captured video data is processed through modules such as pose estimation models, motion analysis models, and cheat detection mechanisms, ensuring reliable and bias-free evaluation.

After input verification, the system performs exercise analysis to compute key performance metrics such as repetitions, flexibility, and strength. The evaluated results are securely stored in a centralized **database**, ensuring **data integrity** and **transparency**. Finally, the system presents the outcomes through an interactive dashboard, displaying performance scores and comparative analytics using **visual graphs**. This architecture ensures scalability, accessibility, and accurate assessment for both **able-users** and **differently-abled users**.

4.1.1 Architecture Diagram

DeepFit AI follows a layered, modular, and **microservices architecture** for automated fitness assessment. The system begins with user authentication (signup/login) and profile validation. Users then perform Height and Weight Estimation followed by Exercise Selection. Video input is captured through live recording or upload, ensuring flexibility and accessibility. The system performs cheat detection using **DeepFake detection**.

Valid inputs are further processed for Exercise Analysis using pose estimation models to accurately track movements and calculate performance metrics. The results are securely stored in database to ensure transparency and tamper-proof record keeping. The dashboard presents performance analytics, **rankings**, and **comparison graphs** in an interactive manner. Additionally, the system supports inclusivity through:

- Blind Assessment (audio-based tests)
- Deaf Assessment (visual/physical tests)

Architecture Diagram

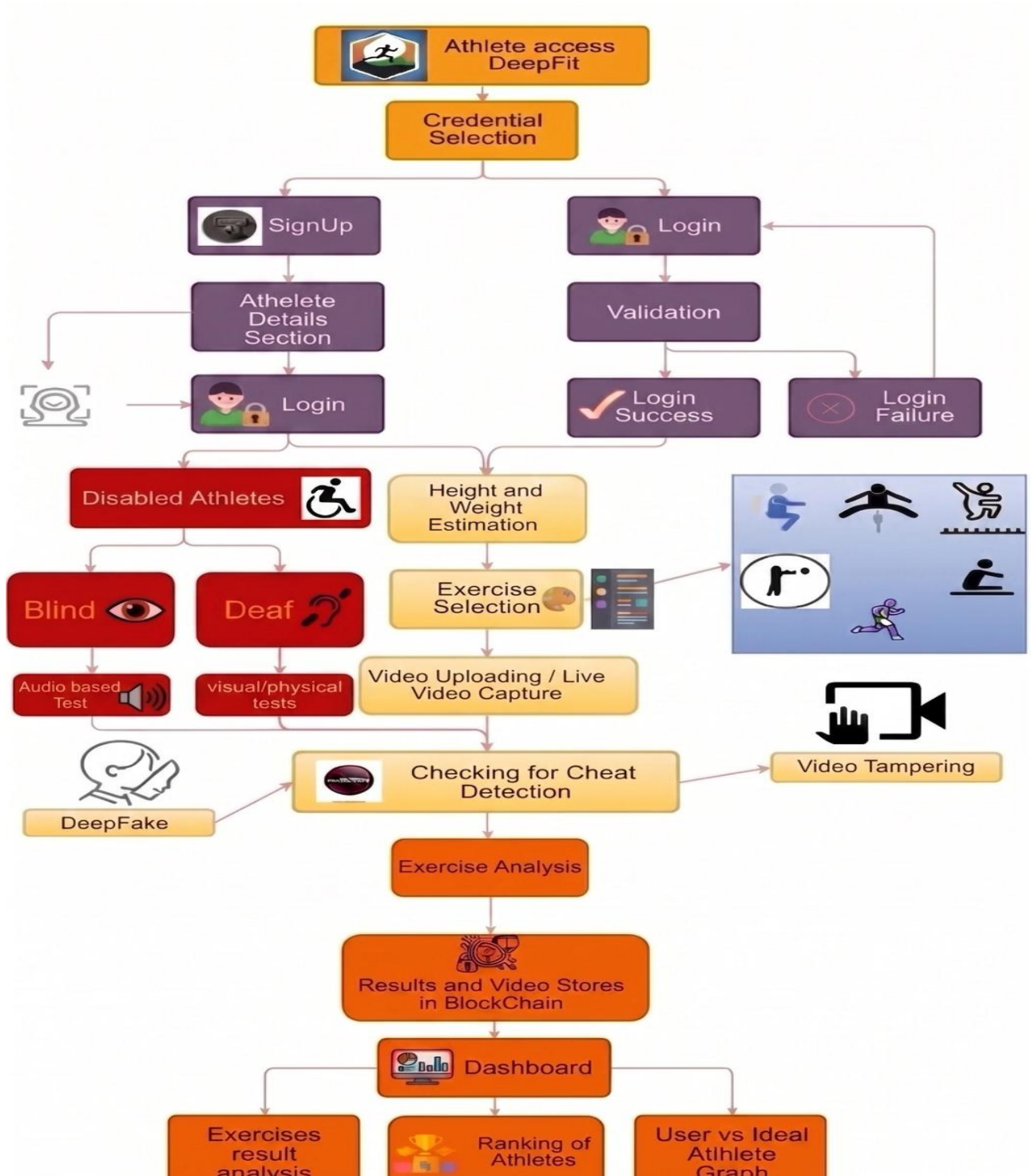


Fig 4.1.1: Architecture Diagram

4.1.2 Use-Case Diagram

The **Fig 4.1.2** describes the use-case diagram represents the interaction between three classes of actors: **Athlete (General User)**, **Disabled Athlete (Blind & Deaf)**, and **Admin** with the DeepFit AI system. It highlights key functionalities such as fitness assessment, video processing, cheat detection, result analysis, and inclusive evaluation through audio-based and visual-based tests. The diagram also includes interactions like exercise selection, real-time video capture, performance analysis, and dashboard visualization.

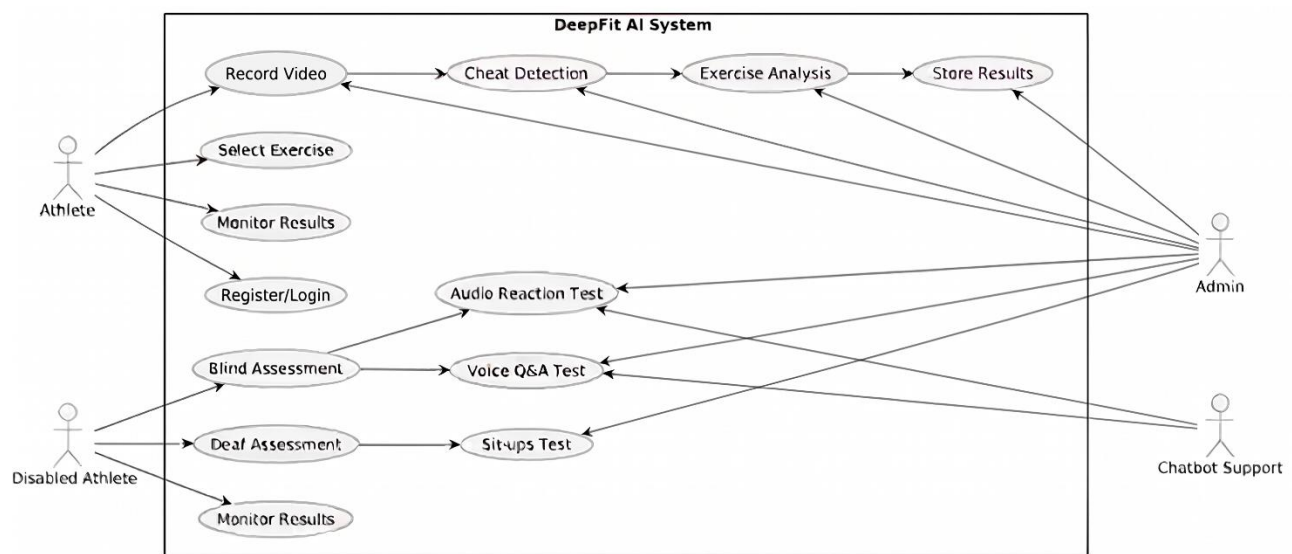


Fig 4.1.2: Use-Case Diagram

4.1.2.1 Use-Case Descriptions

UC-01: User Registration and Login

Use Case ID	UC-01
Actor	Athlete, Disabled Athlete
Description	Register and login to access the system
Main Flow	User enters details → System validates → Access granted

UC-02: Perform Fitness Assessment

Use Case ID	UC-02
Actor	Athlete
Description	Perform exercises like Sit-ups, Vertical Jump, Sit-and-Reach, Dumbbell Curls
Main Flow	Select exercise → Capture/Upload video → System analyzes performance

UC-03: Video Capture and Upload

Use Case ID	UC-03
Actor	Athlete
Description	Provide input video for analysis
Main Flow	User records live video or uploads file → System receives input

UC-04: Cheat Detection

Use Case ID	UC-04
Actor	Admin
Description	Detect fraudulent inputs
Main Flow	Input video → Perform DeepFake Detection → Perform Video Tampering Detection

UC-05: Exercise Analysis

Use Case ID	UC-05
Actor	Admin
Description	Analyze movements using AI
Main Flow	Extract frames → Apply pose detection → Calculate repetitions & metrics

UC-06: Blind Assessment

Use Case ID	UC-06
Actor	Blind Athlete
Description	Perform audio-based evaluation
Main Flow	Play audio instructions → Conduct Audio Reaction Test → Conduct voice-based Q&A

UC-07: Deaf Assessment

Use Case ID	UC-07
Actor	Deaf Athlete
Description	Perform visual-based evaluation
Main Flow	Display instructions → Perform Sit-ups → Analyze via pose detection

UC-08: View Results Dashboard

Use Case ID	UC-08
Actor	Athlete, Disabled Athlete
Description	View performance analytics
Main Flow	Open dashboard → Display scores, graphs, rankings

UC-09: Store Results

Use Case ID	UC-09
Actor	Admin
Description	Store results securely
Main Flow	Save data in database and blockchain

4.1.3 All UML Diagrams

Class-Diagram

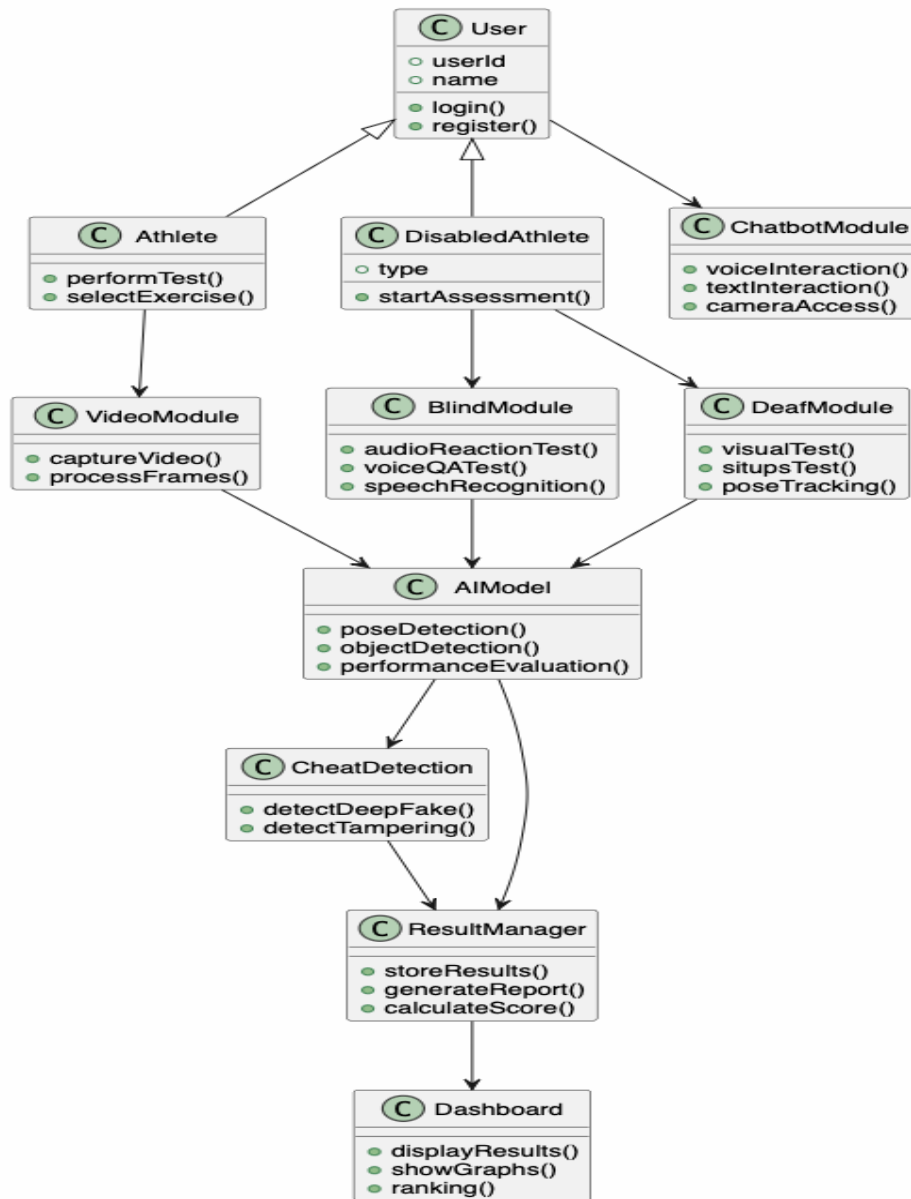


Fig 4.1.3.1: Class Diagram

The Fig-4.1.3.1 describes Class Diagram defines the structure of the system by showing classes, attributes, and methods. It represents relationships between entities like User, Athlete, AI Module, and Result Manager. This diagram helps in designing the object-oriented architecture of the system.

Sequence-Diagram

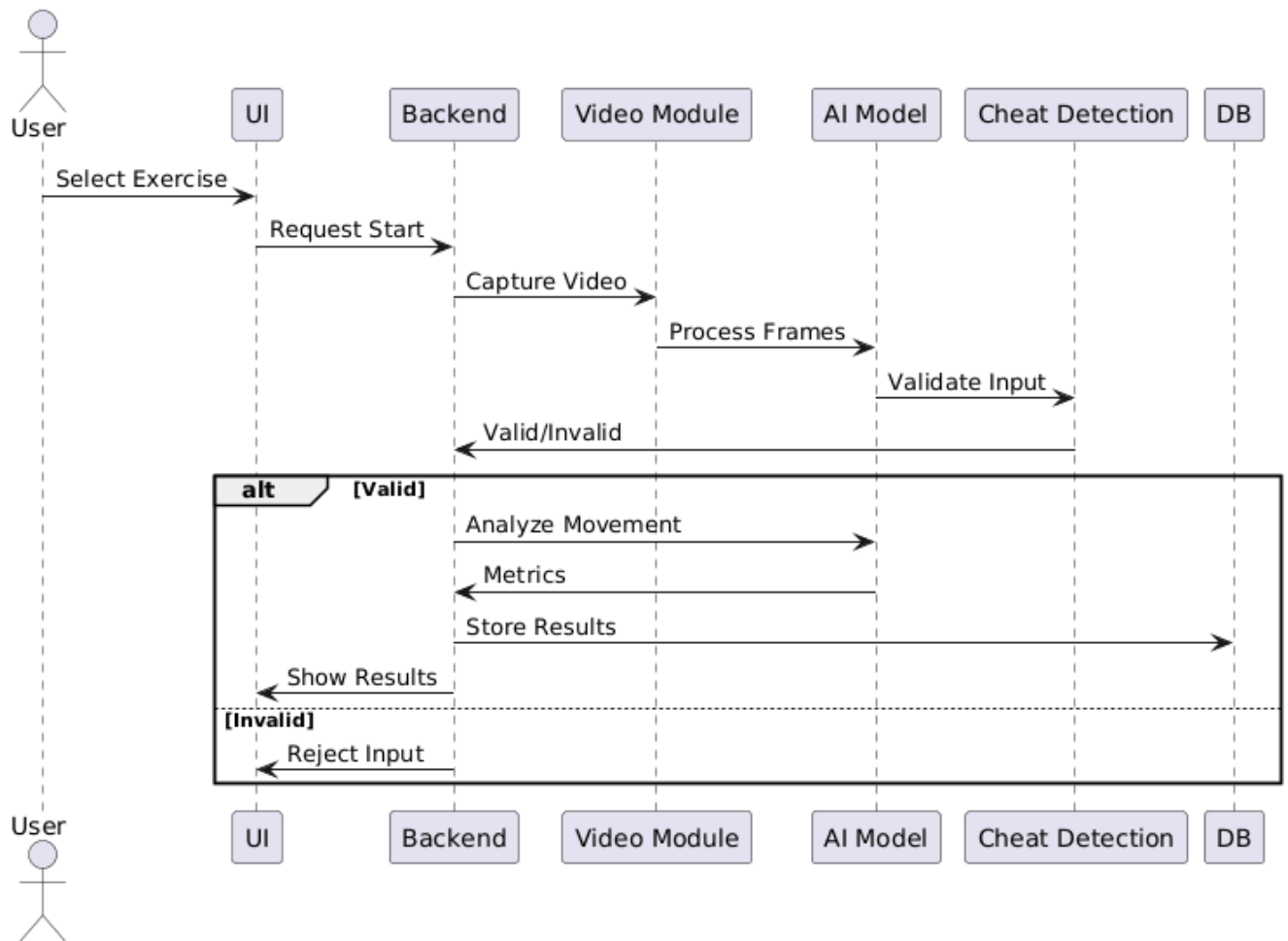


Fig 4.1.3.2: Sequence Diagram

The **Fig 4.1.3.2** describes Sequence Diagram shows the interaction between system components in a time-ordered manner. It represents how requests flow from the user to backend, AI modules, and database. This helps in understanding dynamic behavior and communication between modules.

Activity-Diagram

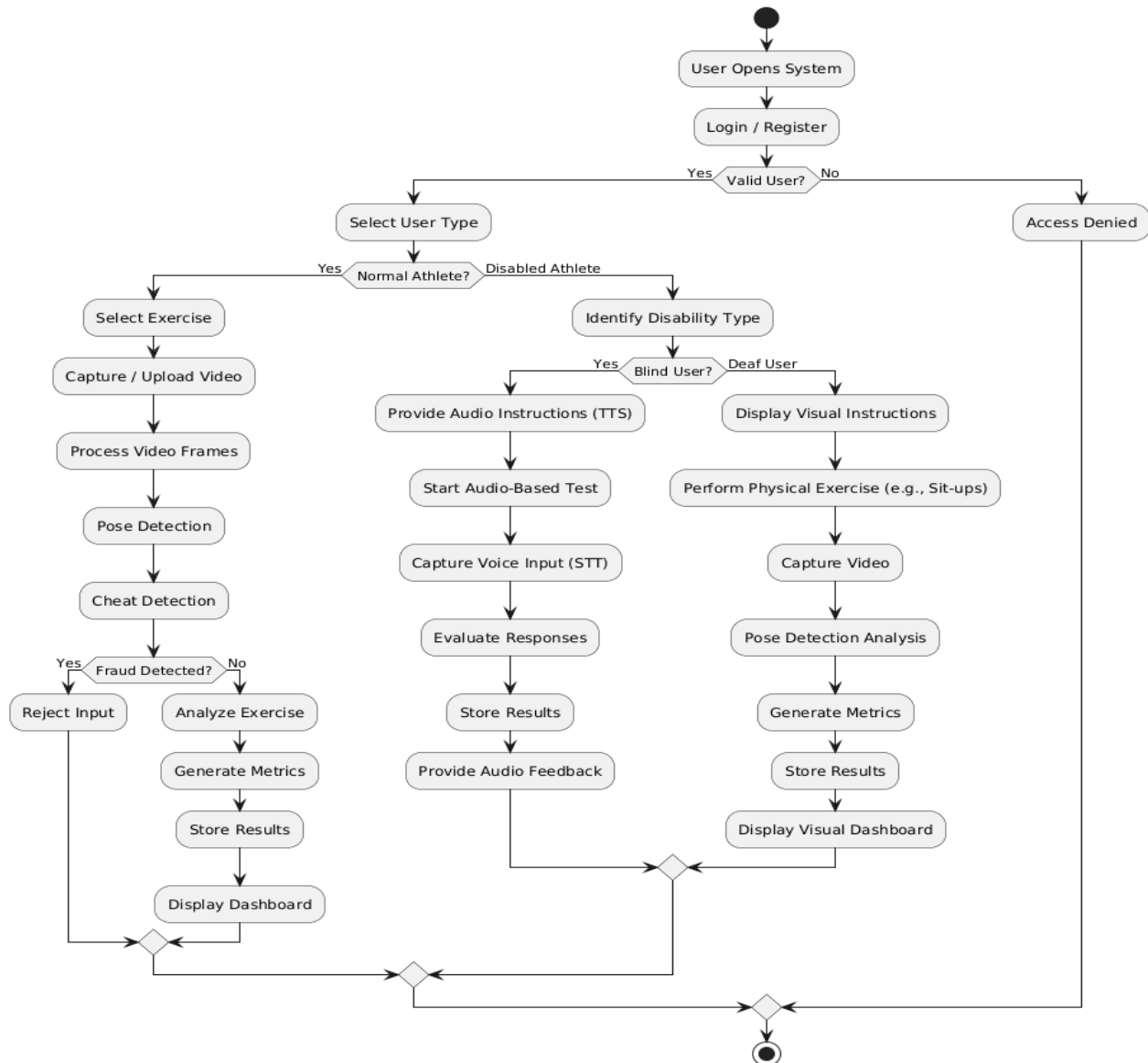


Fig 4.1.3.3: Activity Diagram

The Fig 4.1.3.3 describes Activity Diagram illustrates the workflow of the system from user login to result generation. It shows the sequence of actions including video processing, cheat detection, and analysis. It also includes separate flows for blind and deaf users to ensure inclusivity.

Collaboration-Diagram

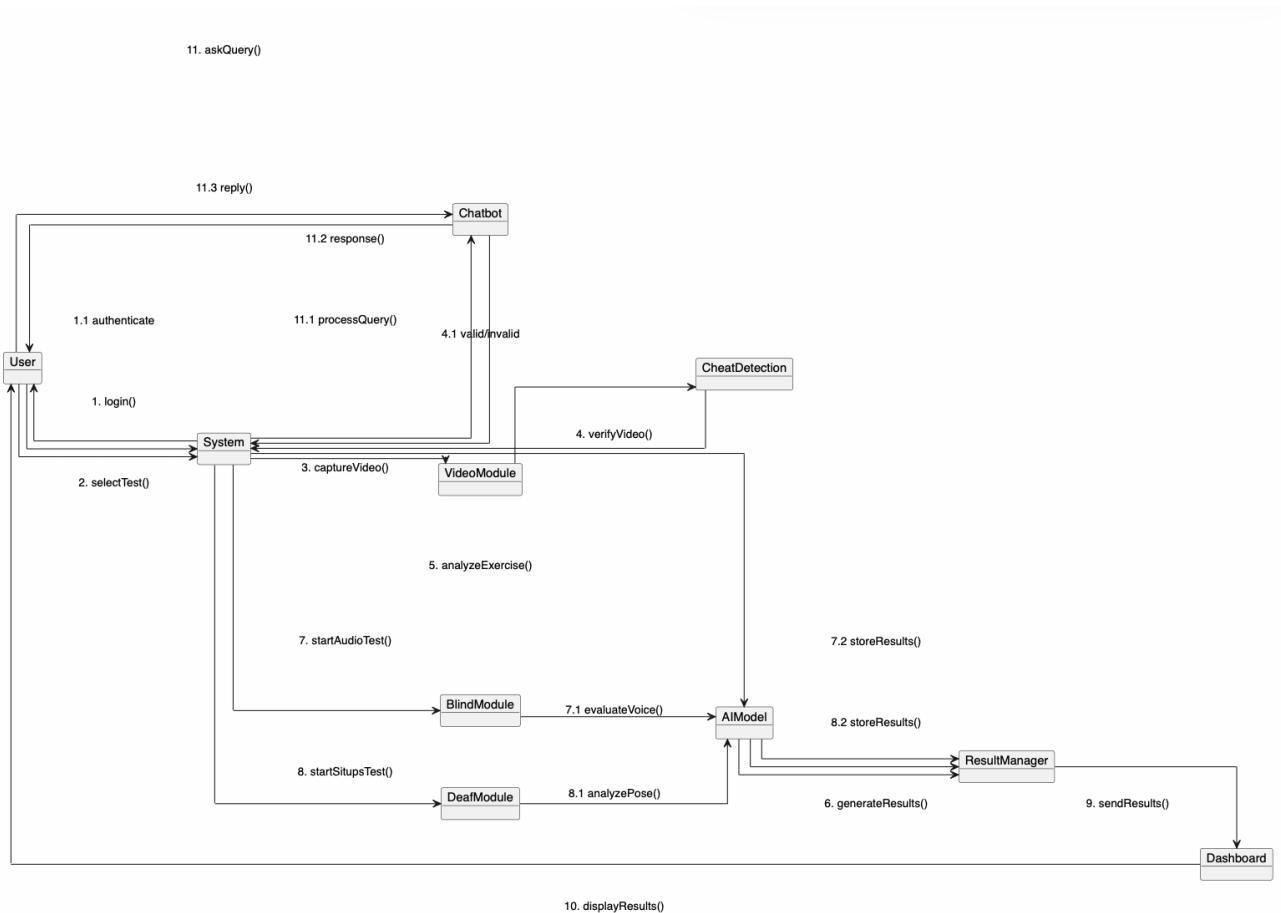


Fig 4.1.3.4: Collaboration diagram

Fig 4.1.3.4 illustrates the collaboration between various modules of the DeepFit AI system during the fitness assessment process. The diagram shows how the user interacts with the system for login, exercise selection, and video submission, while modules such as VideoModule, BlindModule, DeafModule, AIModel, and CheatDetection work together to analyse performance and validate input data. The chatbot provides user assistance, and the ResultManager processes and stores evaluation results before displaying them on the dashboard. This collaboration ensures smooth communication between components for accurate, secure, and inclusive fitness assessment.

State-Diagram

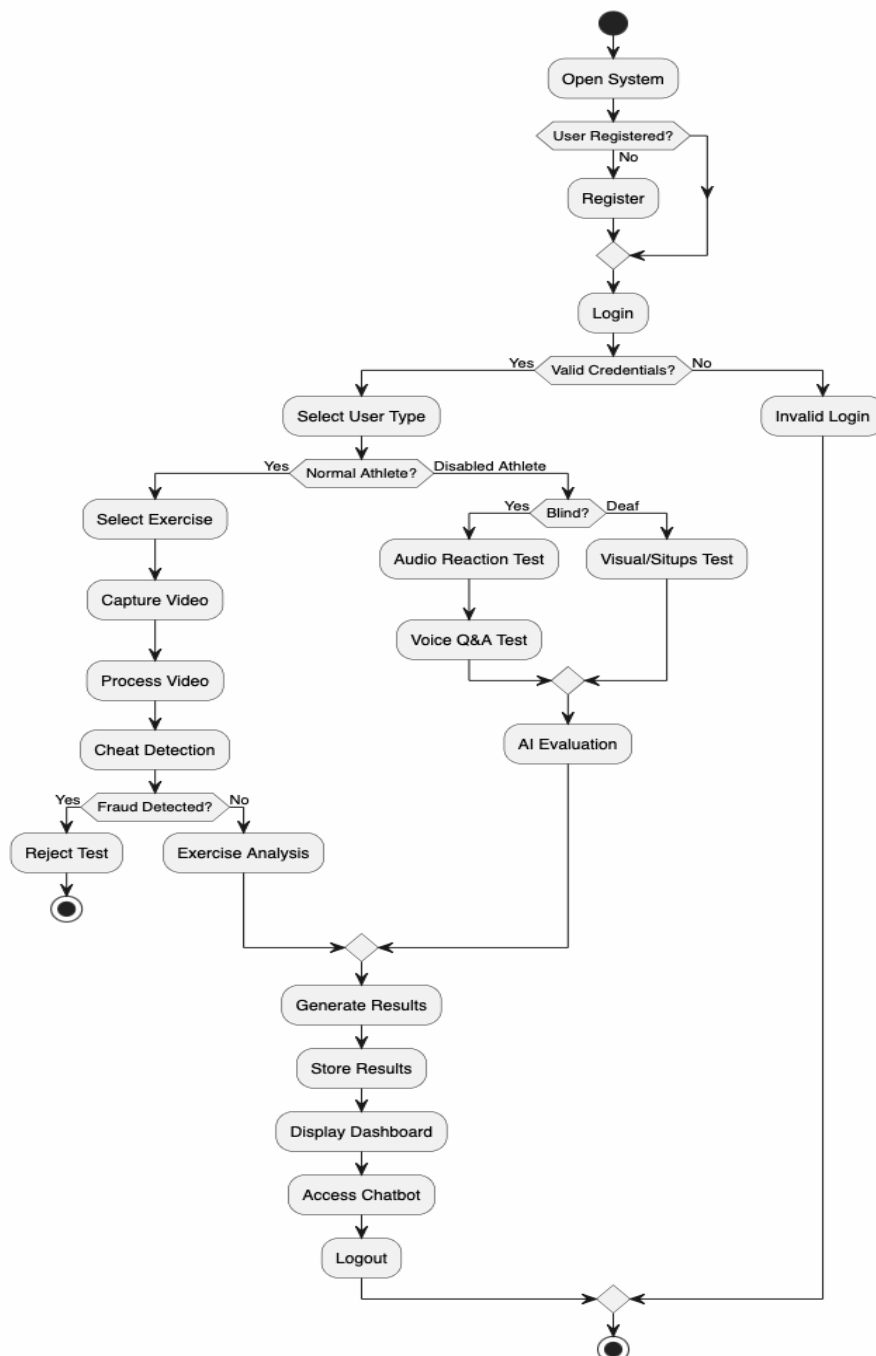


Fig 4.1.3.5: State Diagram

The **Fig 4.1.3.5** describes State Diagram represents different states of the system and transitions between them during execution. It shows how the system moves from login to processing, analysis, and result generation.

Deployment-Diagram

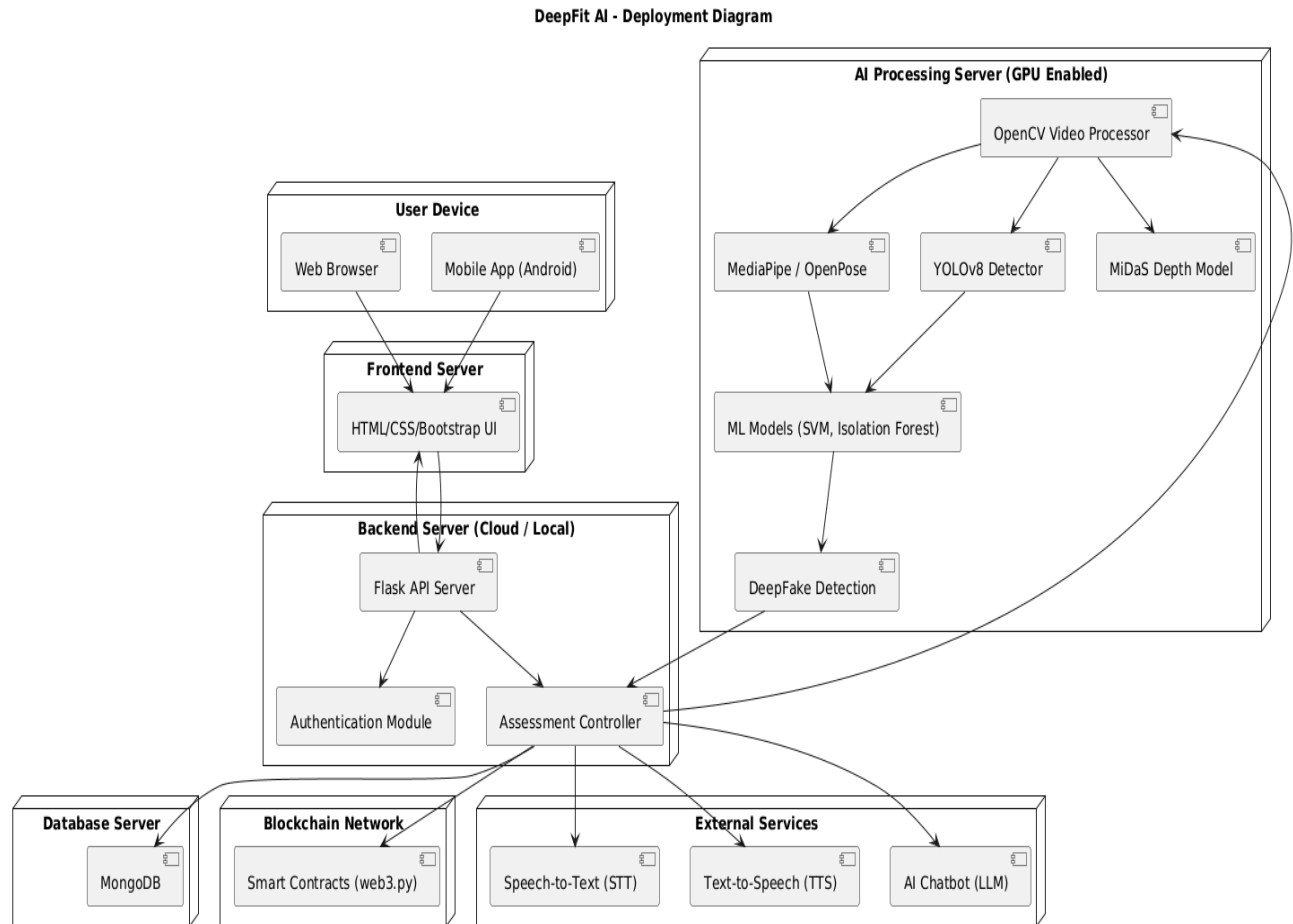


Fig 4.1.3.6: Deployment Diagram

Fig 4.1.3.6 illustrates the deployment architecture of the DeepFit AI system, showing how different hardware and software components interact within the platform. The diagram includes user devices such as web browsers and Android applications connected to the frontend and backend servers for handling user requests and assessment operations. The AI processing server integrates technologies like OpenCV, YOLOv8, MediaPipe/OpenPose, and DeepFake detection models to perform real-time pose estimation, movement tracking, and authenticity verification. External services such as Speech-to-Text, Text-to-Speech, AI chatbot modules, and the MongoDB database are also connected to support secure data management, accessibility features, and intelligent user interaction.

Component-Diagram

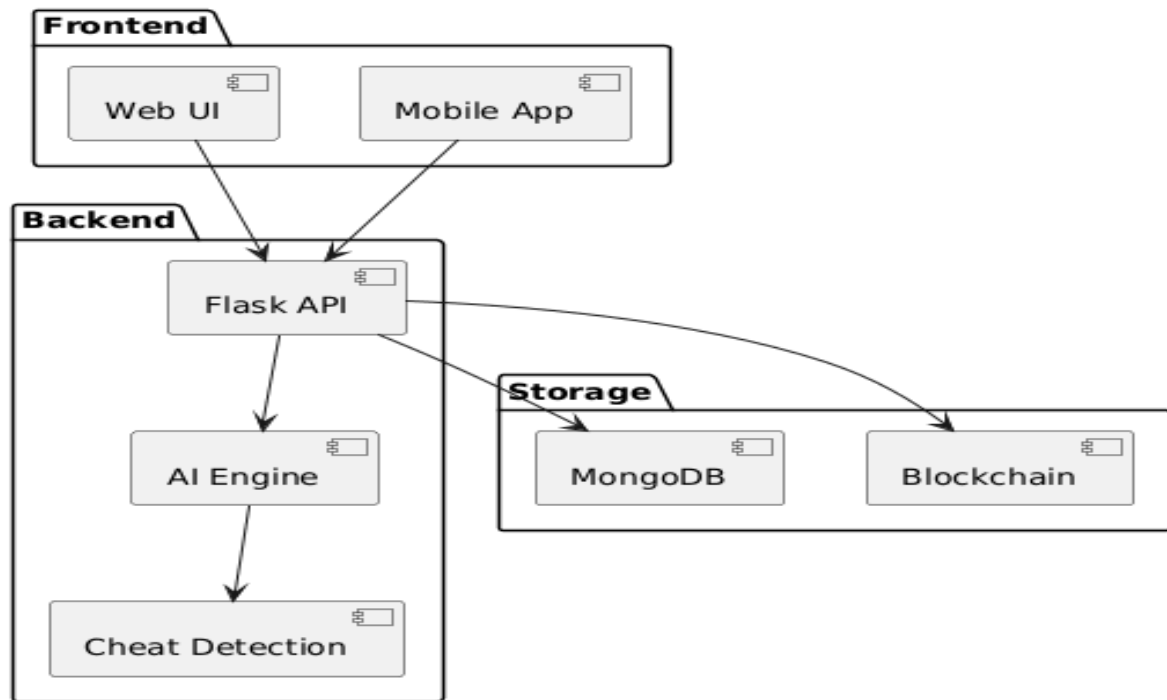


Fig 4.1.3.7: Component Diagram

The **Fig 4.1.3.7** diagram represents major software components like frontend, backend, AI modules, and storage. It explains how different parts of the system interact and integrate.

4.2 Functional Modules

DeepFit AI comprises the following functional modules:

- **Authentication Module**

Handles user registration, login, and validation for secure system access.

- **Profile Management Module**

Stores and manages user details including normal and disabled categories.

- **Fitness Assessment Module**

Performs AI-based evaluation of exercises such as Sit-ups, Vertical Jump, Sit-and-Reach, and Dumbbell Curls.

- **Blind Assessment Module (+audioTest(), +voiceEvaluation())**

Conducts audio-based tests including sound direction detection and voice Q&A evaluation.

- **Deaf Assessment Module (+visualTest(), +poseTracking())**

Performs visual-based evaluation using pose detection for exercises like Sit-ups.

- **Video Processing Module (+captureVideo(), +processFrames())**

Handles video input, frame extraction, and preprocessing.

- **Cheat Detection Module (+detectDeepFake(), +detectTampering())**

Detects fraud using DeepFake Detection and Video Tampering Detection.

- **Result Management Module (+storeResults(), +generateReport())**

Stores results in database and blockchain and generates analytics.

- **Dashboard Module (+displayResults())**

Displays performance metrics, rankings, and comparison graphs.

- **AI Chatbot Module (+assistUser(), +answerQueries())**

Provides user guidance using text, voice, and camera interaction.

4.2.1 Screenshots

The system interface presents a structured dashboard for monitoring and managing the application data. The home screen displays key modules, user options, and system status.

The dashboard provides:

- Overview of system activity and data records
- Interactive sections for user input and data visualization
- Real-time updates (if applicable to your project)

The visualization screen includes:

- Tabular and graphical representation of data
- Interactive elements such as filters, search, and sorting
- Dynamic updates based on user interaction



DeepFit Icon

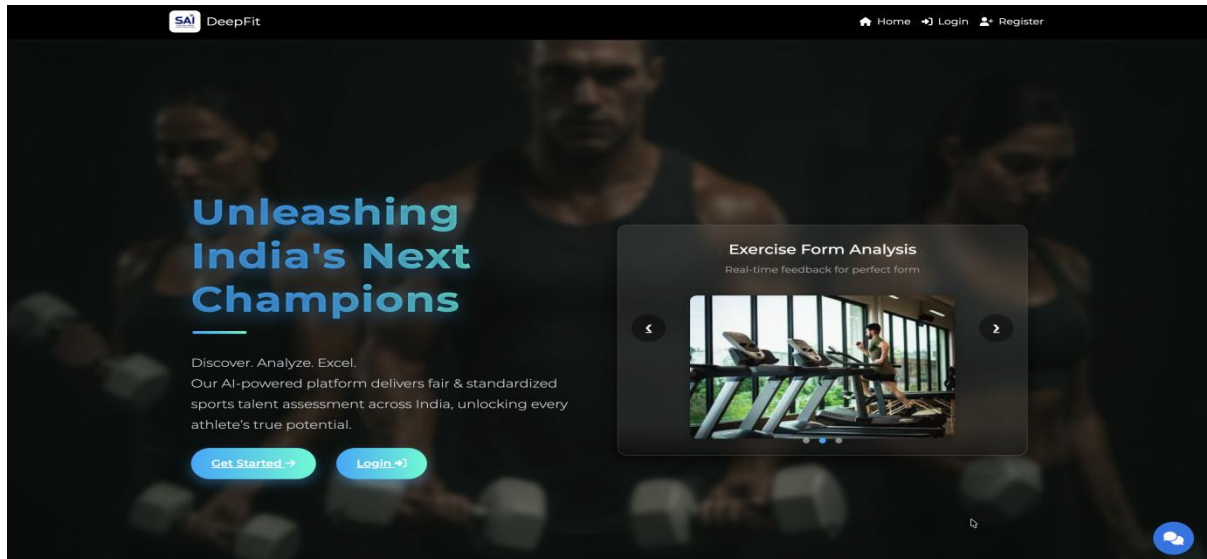


Fig 4.1: Home Page

The **Fig 4.1** screen shows the initial interface of the system with the project logo and user input options. It provides the entry point for accessing the platform functionalities.

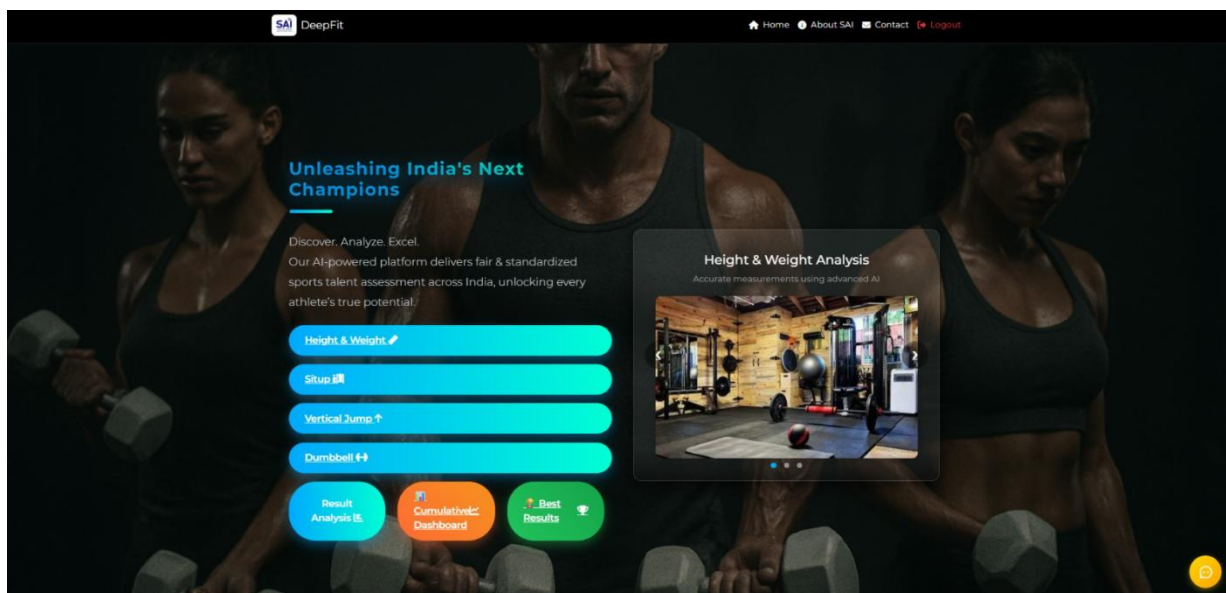


Fig 4.2: Exercise Over View Page

The **Fig 4.2** interface presents the AI-powered fitness assessment platform with multiple test options. It highlights data-driven evaluation and a modern dashboard design.

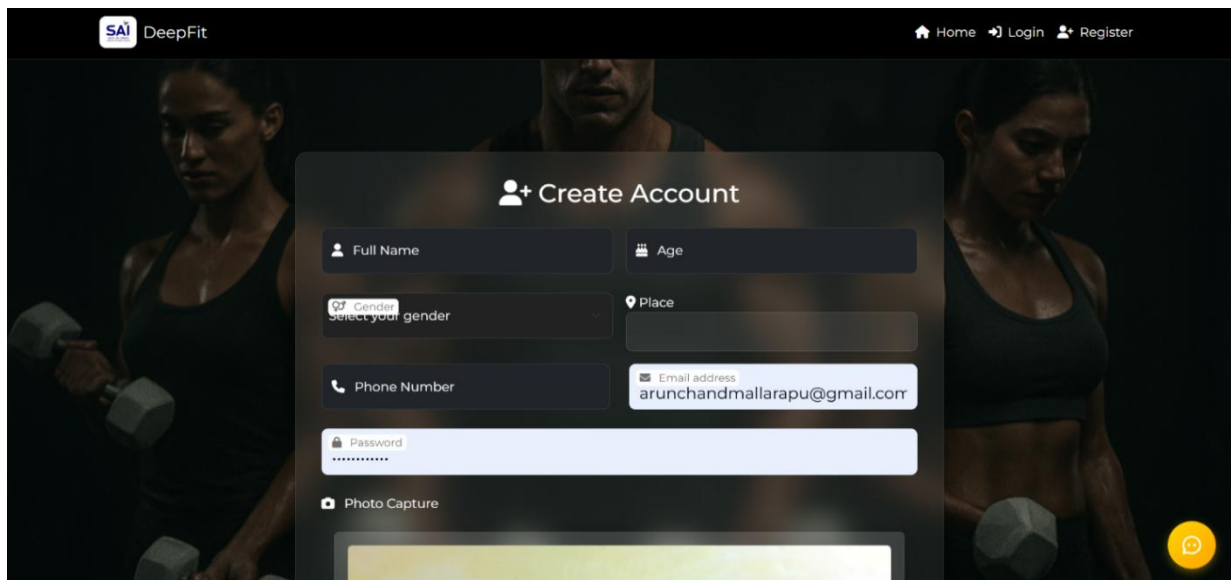


Fig 4.3: Create Account Page

The **Fig 4.3** This screen displays the user registration form with input fields and photo capture feature. It ensures proper user data collection with an engaging UI design.

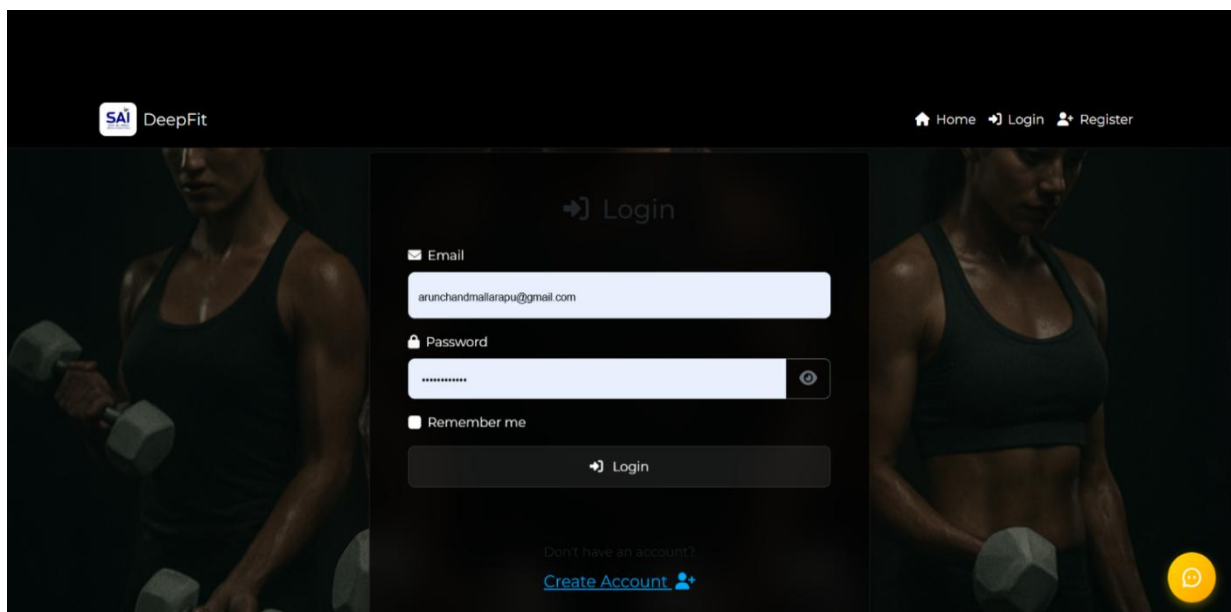


Fig 4.4: Login Page

The **Fig 4.4** is the login interface allowing users to securely access the system using credentials. It features a clean and modern design for easy navigation.

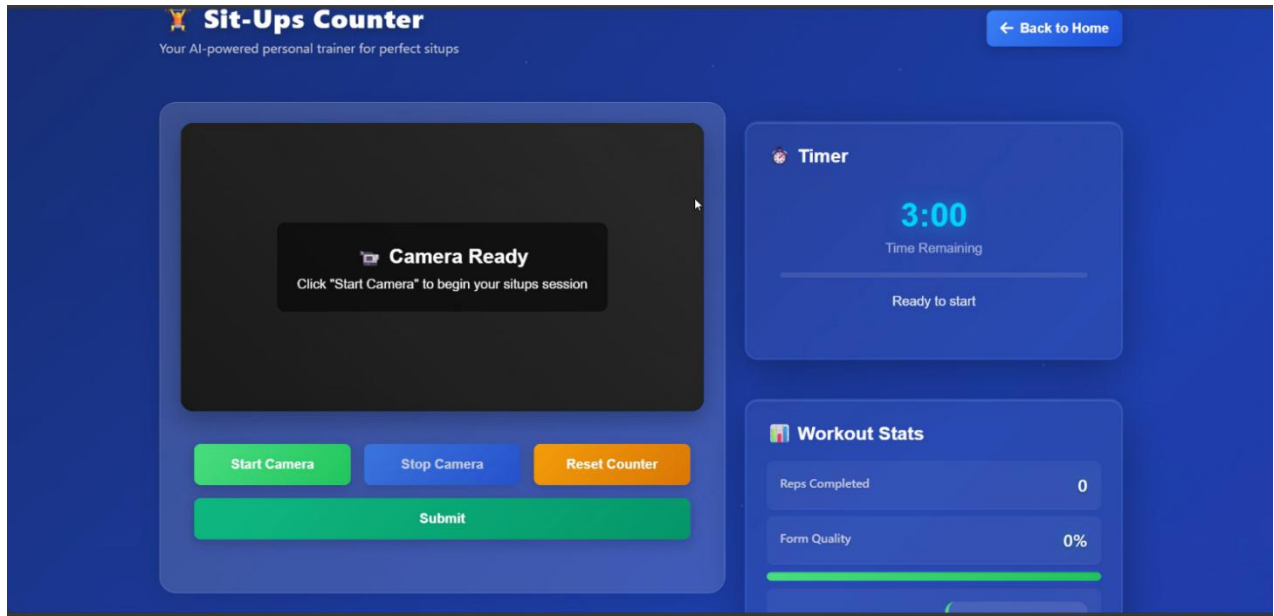


Fig 4.5: Sit Up Interface

The **Fig 4.5** screen shows the sit-ups tracking interface with real-time camera-based monitoring. It displays workout statistics and helps users track performance effectively.

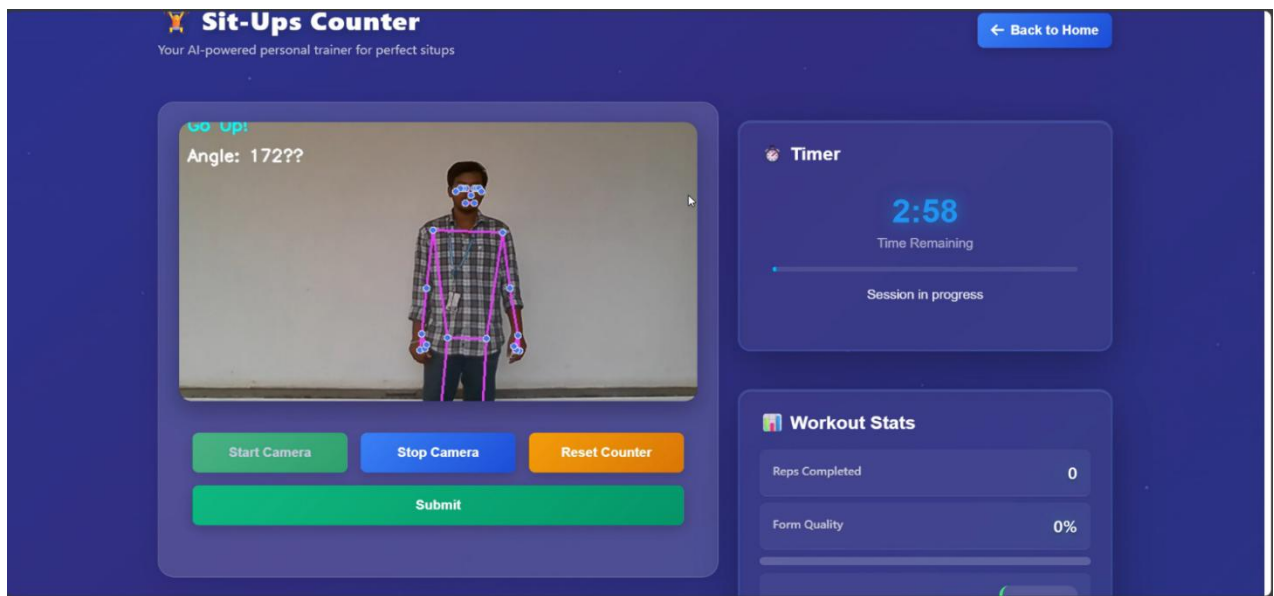


Fig 4.6: Sit Up Analysis

The **Fig 4.6** screen shows real-time pose detection during a sit-ups session with body keypoints and angle tracking. It provides live feedback along with workout stats and a timer.

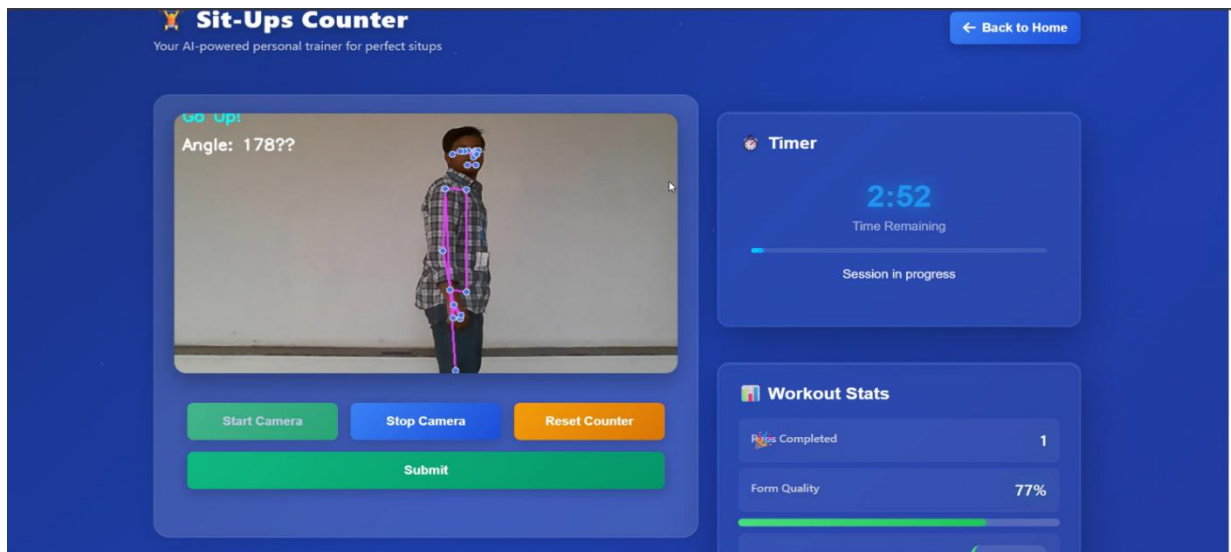


Fig 4.7: Sit Up Calculations

The **Fig 4.7** interface displays AI-based sit-ups tracking with accurate repetition counting using pose estimation. It updates performance metrics like reps and form quality dynamically.



Fig 4.8: Sit Up Results

The **Fig 4.8** dashboard presents sit-up results including reps, form quality, and total workout duration. It provides a clear summary for performance analysis and tracking.



Fig 4.9: Sit Up Performance Overview

The **Fig 4.9** screen shows a performance graph comparing reps and form quality over time. It helps users analyze consistency and improvement across sessions



Fig 4.10: Sit Up Comprehensive Analysis

The **Fig 4.10** dashboard displays detailed performance analysis using a dual-axis graph. It visually compares exercise repetitions with form quality for better insights.



Fig 4.11: Height and Weight Interface

The **Fig 4.11** interface shows AI-based height and weight estimation using computer vision. It tracks body measurements in real time with highlighted body segments.

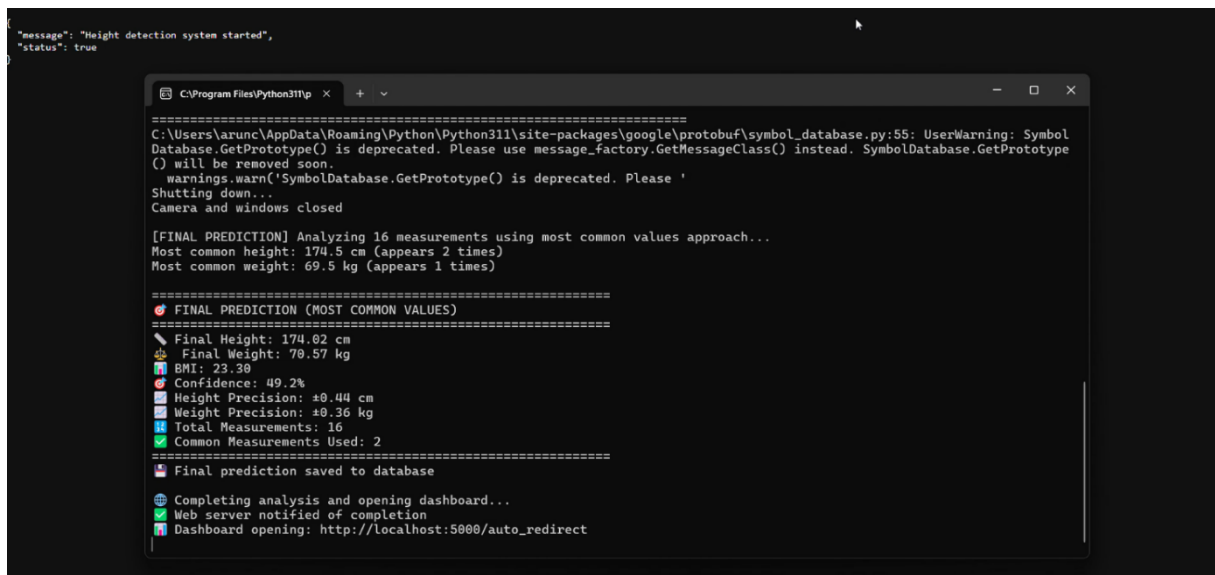


Fig 4.12: Height And Weight Command line Gui

The **Fig 4.12** screen displays the final output of height and weight prediction with calculated values. It also shows system processing details and database storage confirmation.

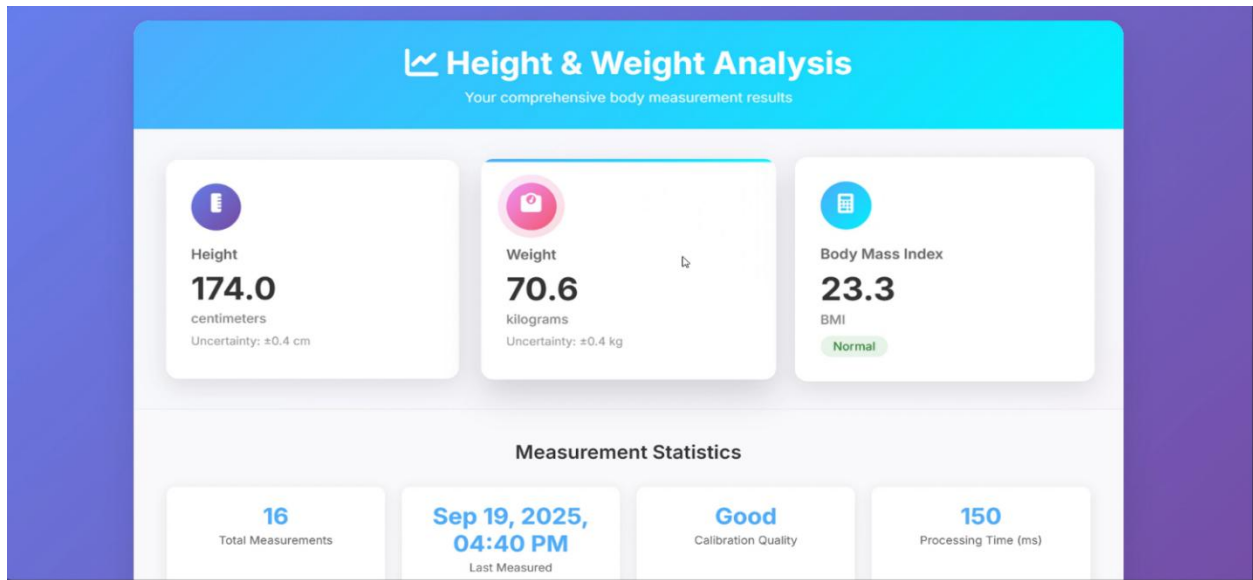


Fig 4.13: Height And Weight Result

The **Fig 4.13** dashboard presents height, weight, and BMI analysis along with system calibration details. It provides accurate health metrics with performance indicators.

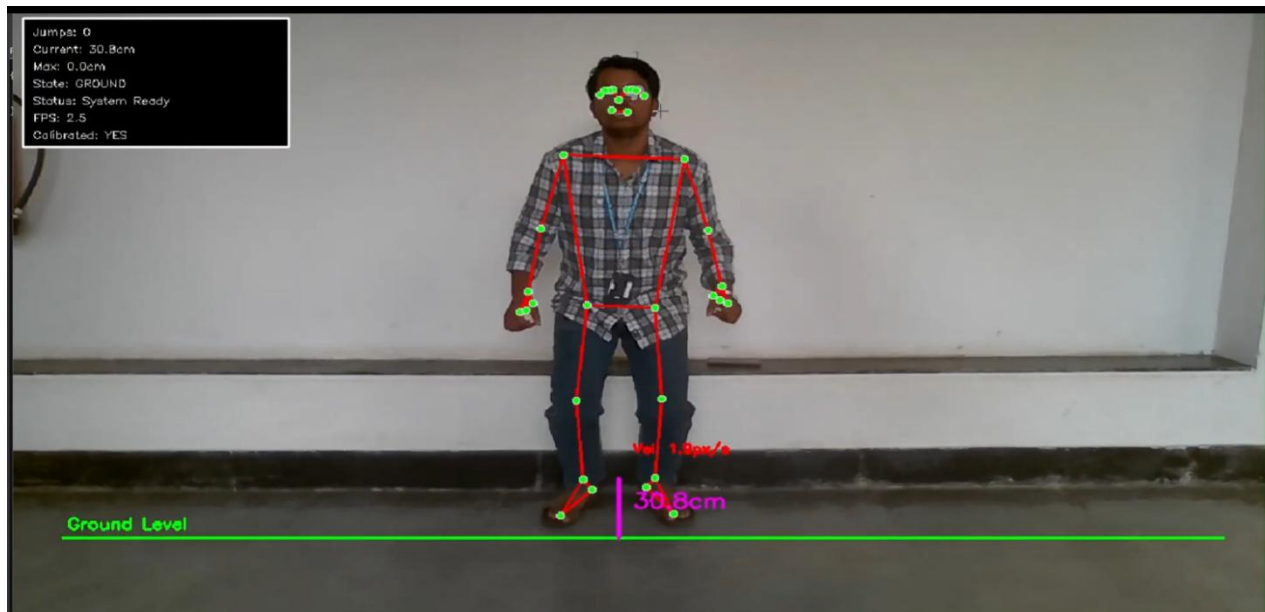


Fig 4.14: Vertical Jump Interface

The **Fig 4.14** interface shows vertical jump tracking using pose estimation and real-time monitoring. It displays jump height, count, and system status.

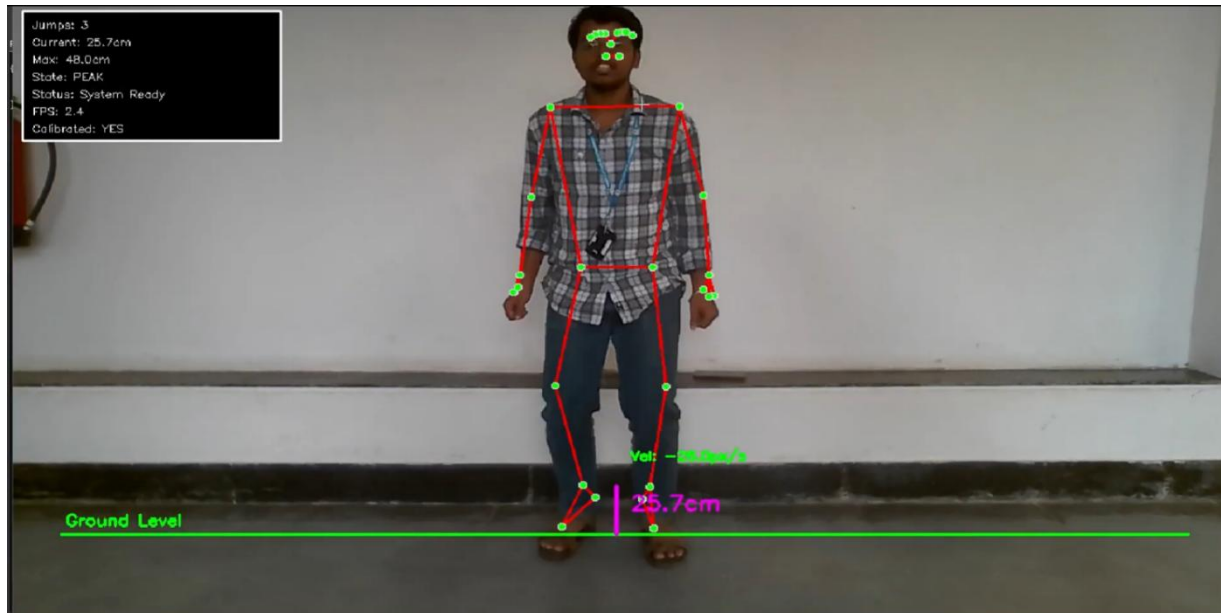


Fig 4.15: Vertical Jump Analysis

The **Fig 4.15** screen captures jump performance with height measurement. It shows peak jump state and maintains stable tracking accuracy.



Fig 4.16: Vertical Jump Results

The **Fig 4.16** This dashboard displays vertical jump results including maximum and average height. It also shows total jumps and session summary.



Fig 4.17: Vertical Jump Analysis

The **Fig 4.17** multi-panel dashboard provides detailed analysis of jump performance and body mechanics. It includes graphs for jump progression and angle comparisons.

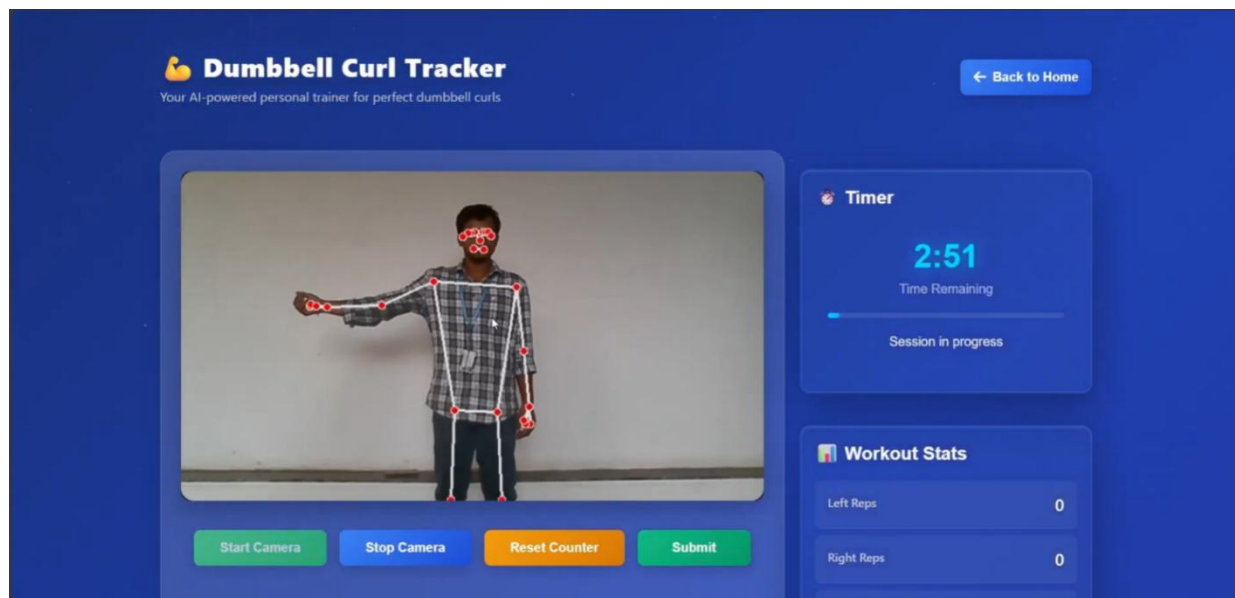


Fig 4.18: Dumbbell Curl Interface

The **Fig 4.18** screen shows the dumbbell curl tracking interface with real-time repetition counting. It monitors exercise performance using pose detection and displays workout stats.

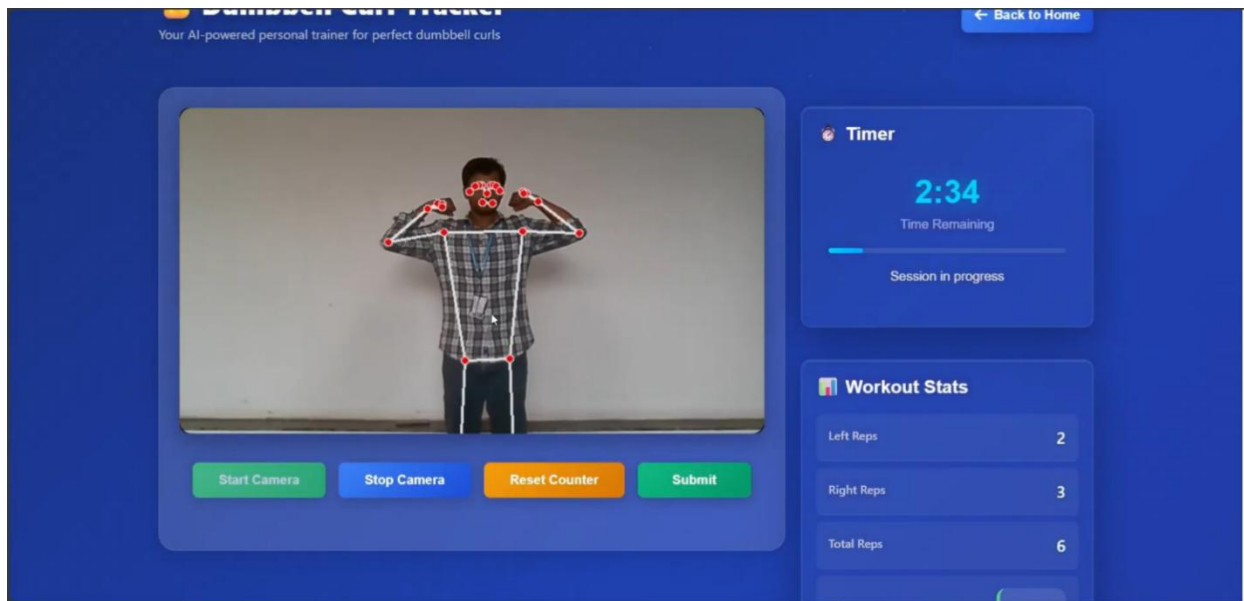


Fig 4.19: Dumbbell curl Calculation

The **Fig 4.19** interface displays live exercise tracking with real-time pose detection. It shows repetitions, timer, and performance statistics during the workout.

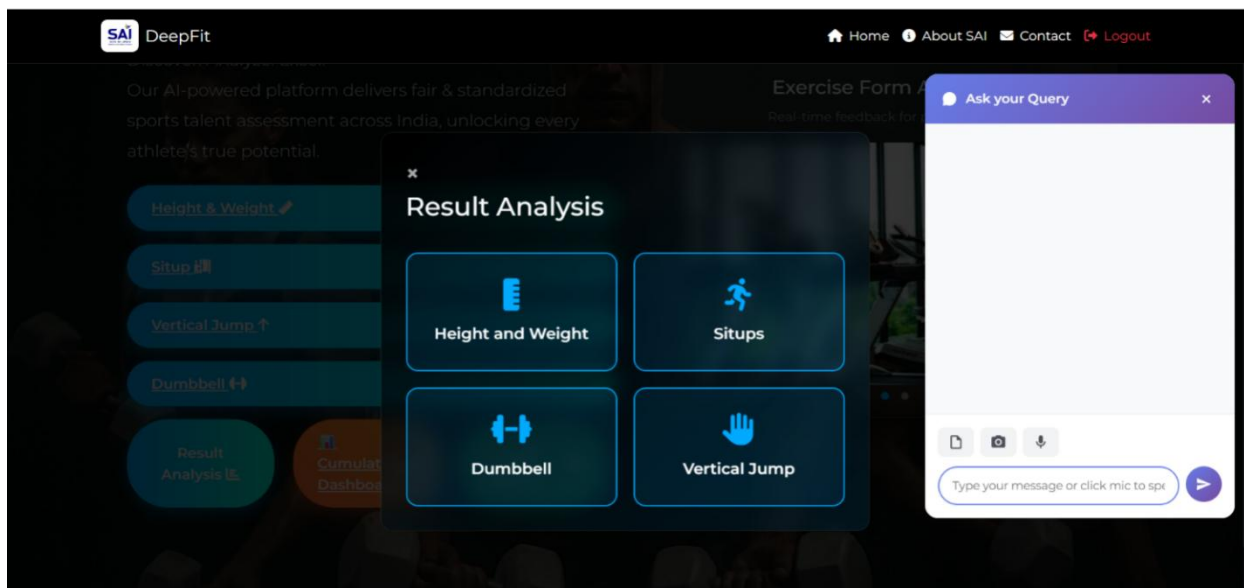


Fig 4.20: Overall Results

The **Fig 4.20** screen presents the result analysis module integrated with an AI chatbot. It allows users to view results and interact with the chatbot for assistance.

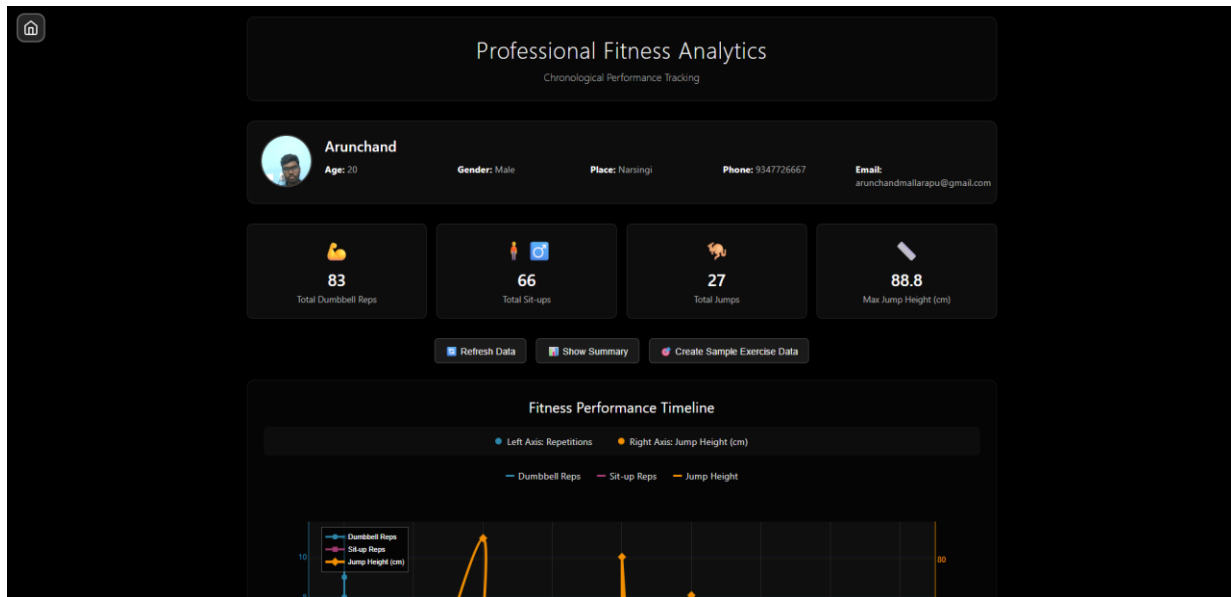


Fig 4.21: Personal Dashboard

The **Fig 4.21** dashboard displays detailed fitness analytics including user profile and performance metrics. It uses graphs to track progress and evaluate overall performance trends.

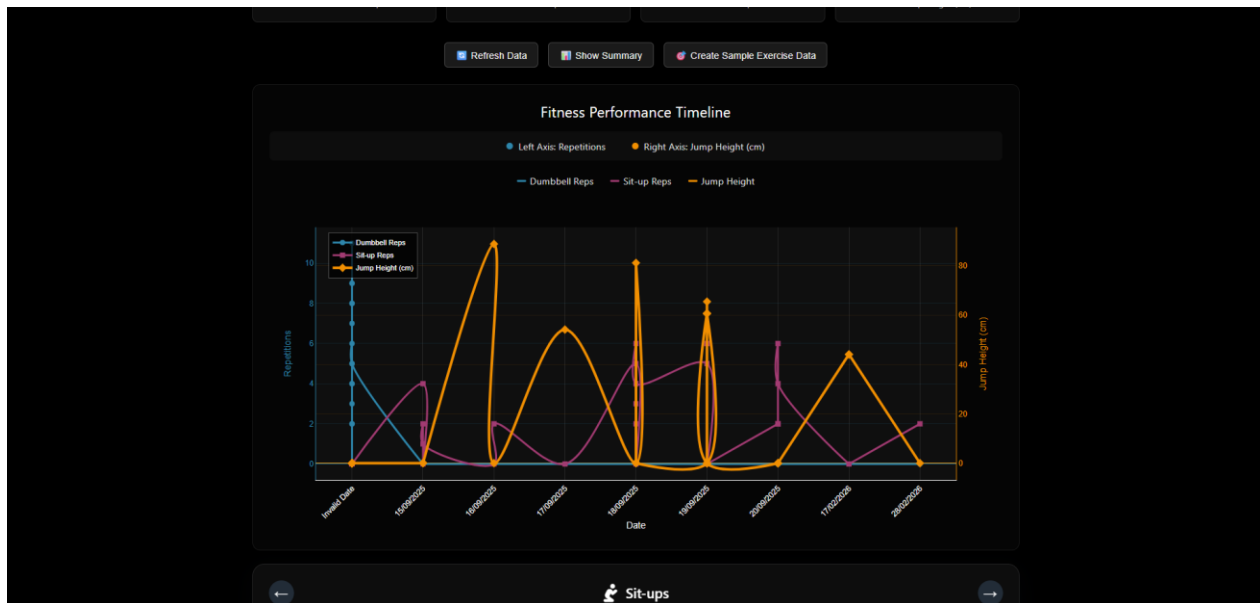


Fig 4.22: Overall Exercise Analysis

The **Fig 4.22** screen shows a performance timeline graph illustrating variations across different tests. It helps analyze consistency and improvement over time.

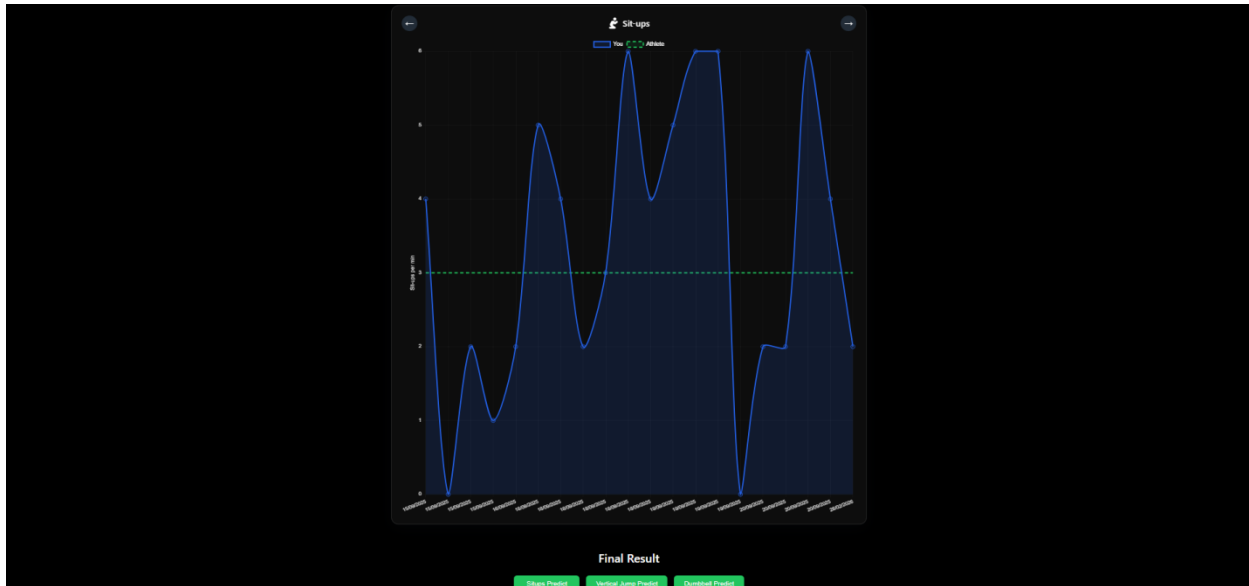


Fig 4.23: Exercise Bench Mark

The **Fig 4.23** graph presents final performance analysis with detailed score visualization. It highlights peaks, drops, and overall performance stability.

Best Results - Qualified Athletes
Top performers ranked by average score and completion time

Rank	Name	Time	Staps	Height/Weight	Average Score
1	Arunchand	00:03:00	7	174.9cm / 60.3kg	26.54
2	Nathan	00:03:00	6	174cm / 70.6kg	25.32
3	Nathan	00:03:00	5	174cm / 70.6kg	23.84
#4	Arunchand	00:03:00	4	174.9cm / 60.3kg	20.37
#5	Arunchand	00:03:00	4	174.9cm / 60.3kg	17.36

Fig 4.24: Athelete Dashboard

The **Fig 4.24** leaderboard displays rankings of top athletes based on scores and qualification status. It enables comparison among users.

Emulator Mobile App images (Results):



Fig 4.25: DeepFit App Logo

The **Fig 4.25** screen describes the DeepFit AI platform with real-time fitness and health monitoring features. It uses computer vision for accurate performance analysis and feedback.

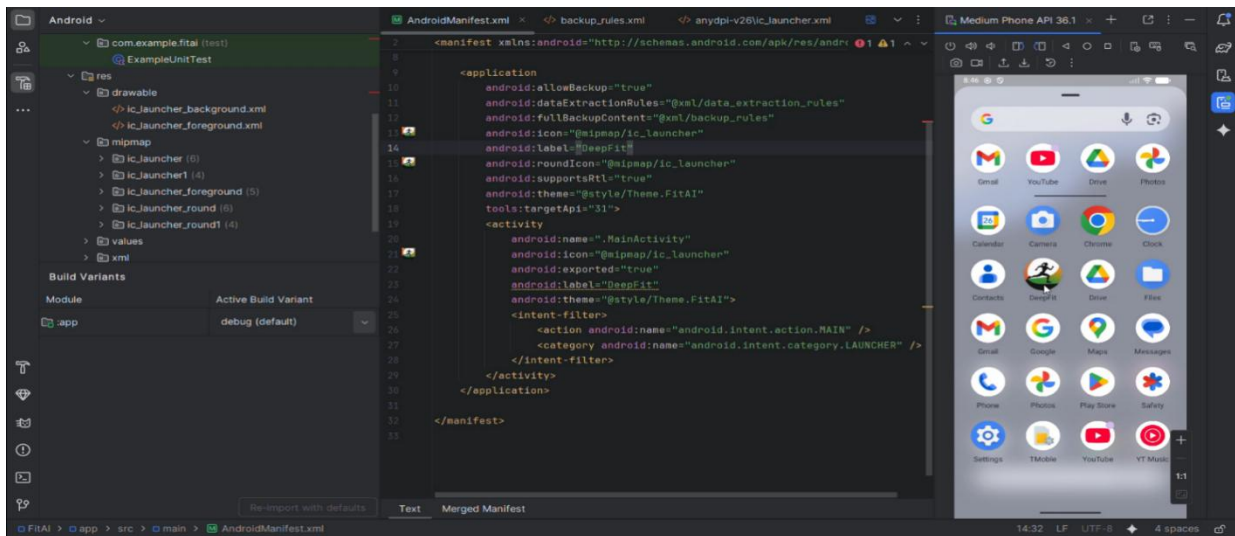


Fig 4.26: Mobile Emulator Interface

The **Fig 4.26** screen shows the development environment with Android code implementation. It also includes a mobile preview for user interaction.

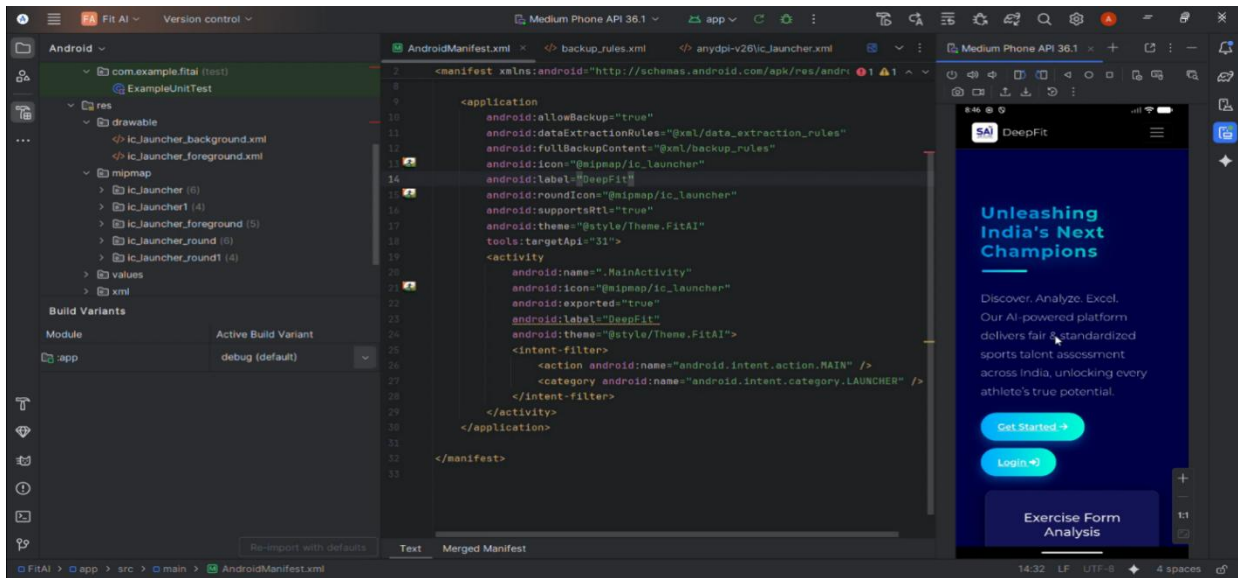


Fig 4.27: Mobile Emulator Home Page

The Fig 4.27 interface displays a welcome screen with introduction content and navigation options. It provides an entry point for users into the application.

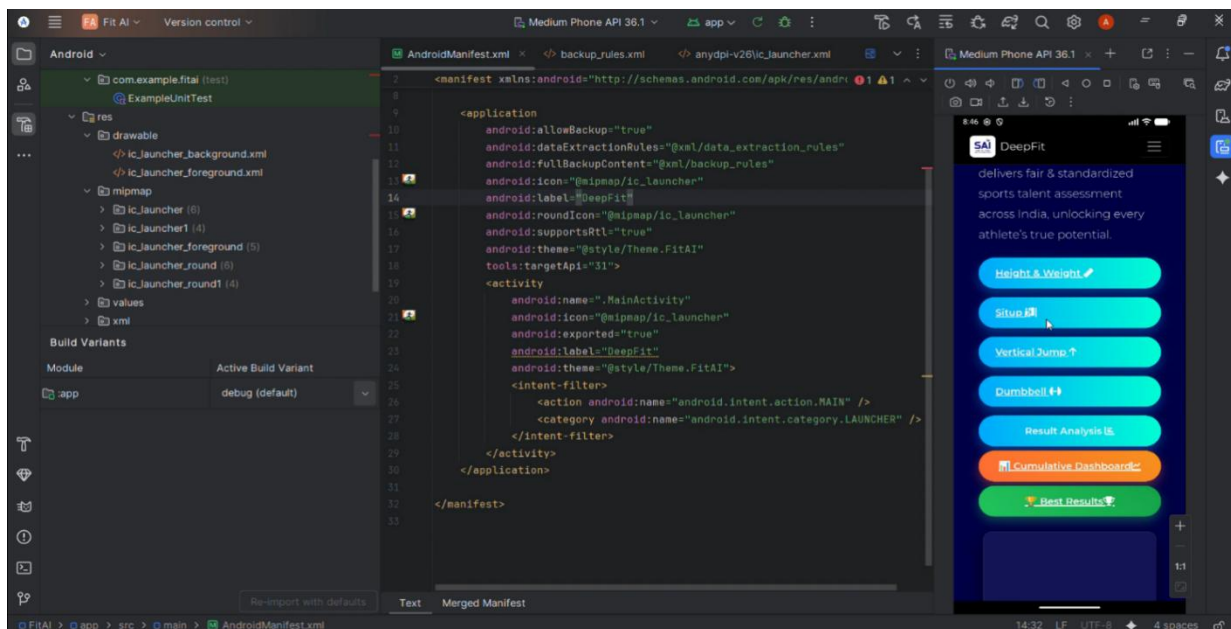


Fig 4.28: Mobile Emulator Exercise Overview Page

The Fig 4.28 screen shows multiple selectable options in a mobile preview layout. It allows users to navigate different features of the application.

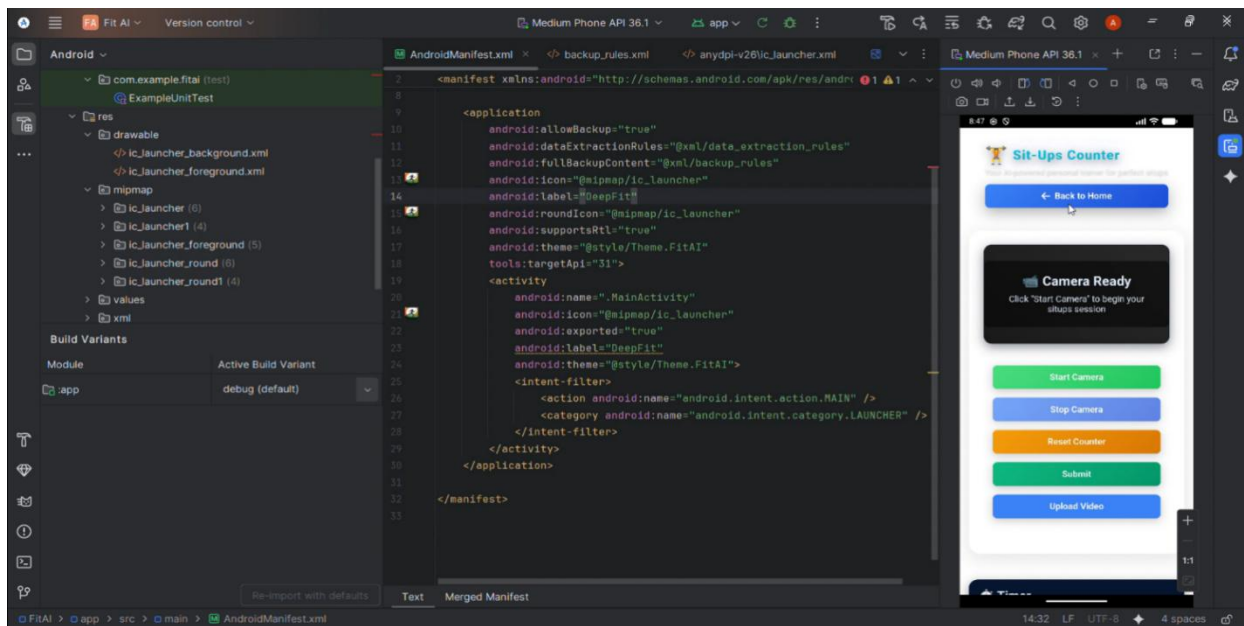


Fig 4.29: Mobile Emulator Sit Up Interface

The Fig 4.29 screen displays Android application code along with a mobile preview. It highlights interface design and functionality implementation of situps.

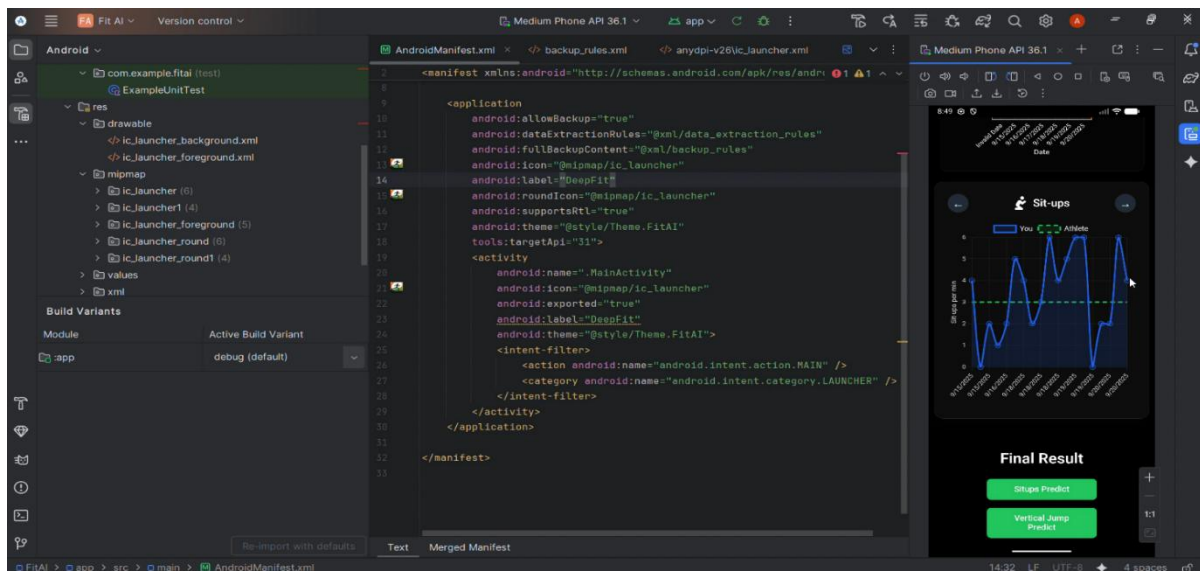


Fig 4.30: Mobile Emulator Exercise Bench Mark Page

The Fig 4.30 This interface shows coded UI components with performance graphs and result sections. It represents data visualization and evaluation output.

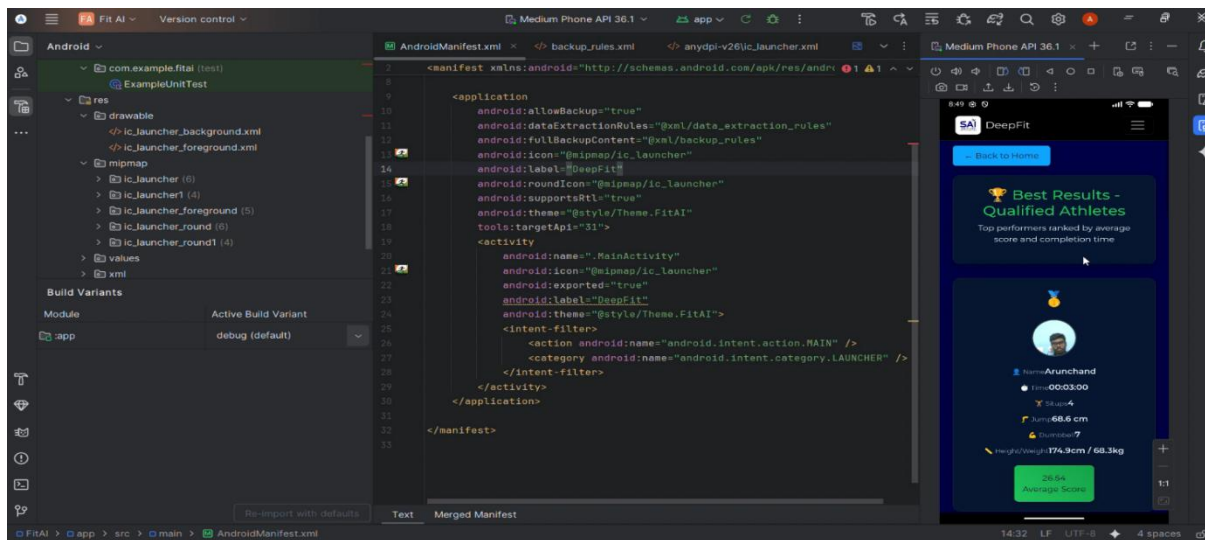


Fig 4.31: Mobile Emulator Athlete Dashboard Page

The **Fig 4.31** screen displays a mobile preview highlighting best results and qualified athletes. It showcases final output with achievements and rankings.

Blind and Deaf Screenshots:

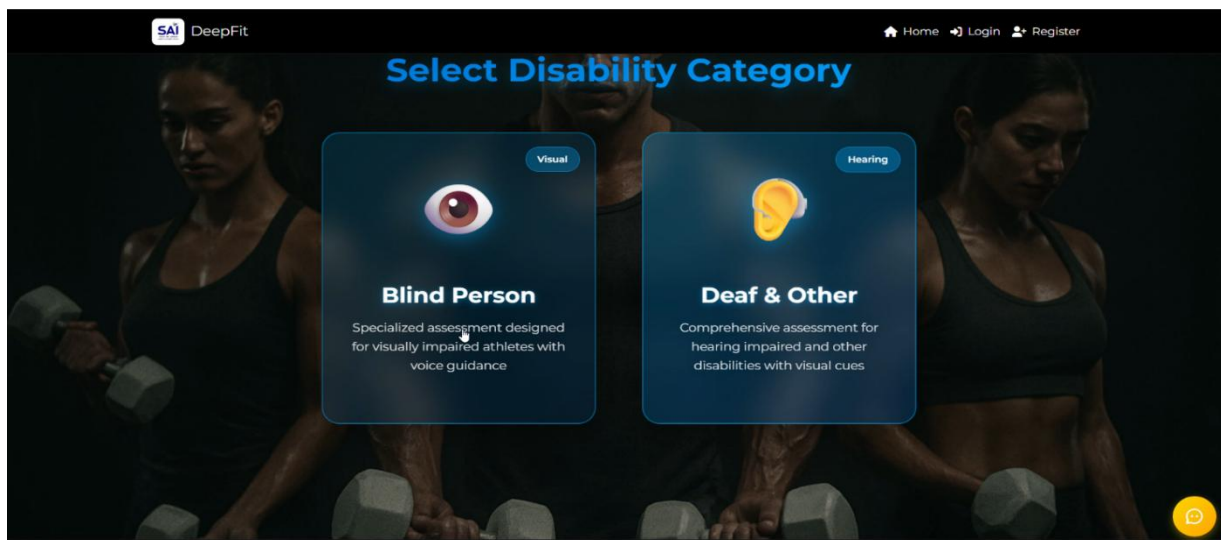


Fig 4.32: Blind And Deaf&Other Interface

The **Fig 4.32** interface allows users to select disability categories like blind or deaf. It provides accessible navigation using a clean card-based layout.

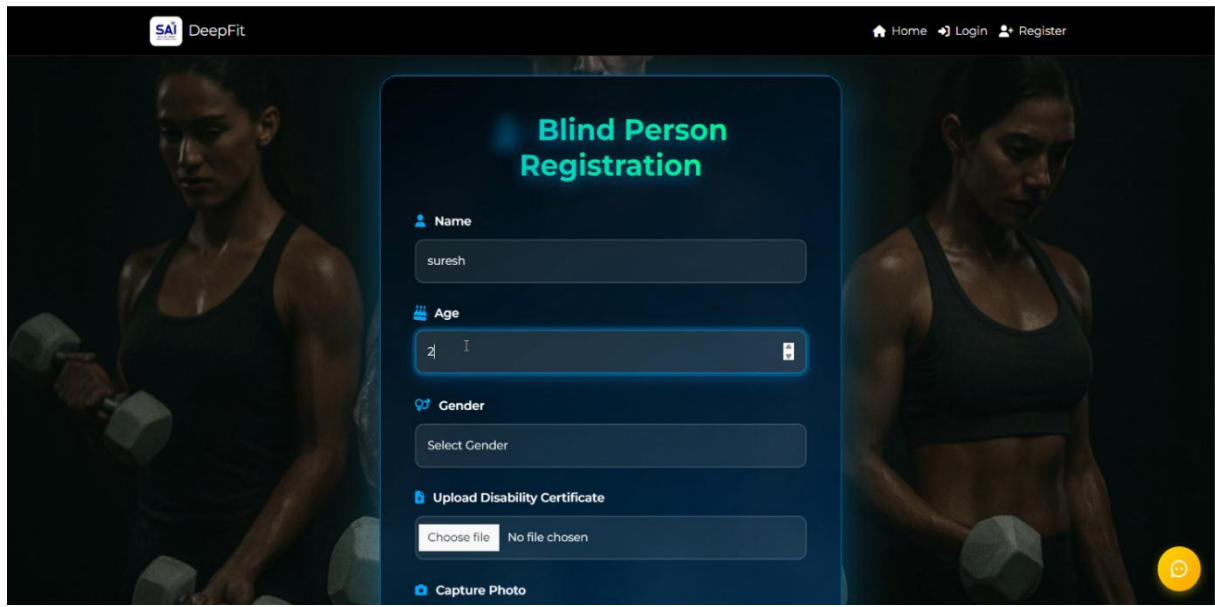
The image shows a web application interface for "Blind Person Registration". The header includes the "SAI DeepFit" logo and navigation links for Home, Login, and Register. The main content area is a dark blue panel with the title "Blind Person Registration" in green. It contains several input fields: "Name" with the value "suresh", "Age" with the value "21", and "Gender" with a dropdown menu showing "Select Gender". There is also an "Upload Disability Certificate" section with a "Choose file" button and the text "No file chosen". At the bottom of the panel is a "Capture Photo" button. The background of the page features a woman in a black sports bra and leggings, holding a dumbbell.

Fig 4.33: Blind Person Registration Page

The **Fig 4.33** screen shows a registration form for visually impaired users with input fields and certificate upload. It ensures proper user data collection.

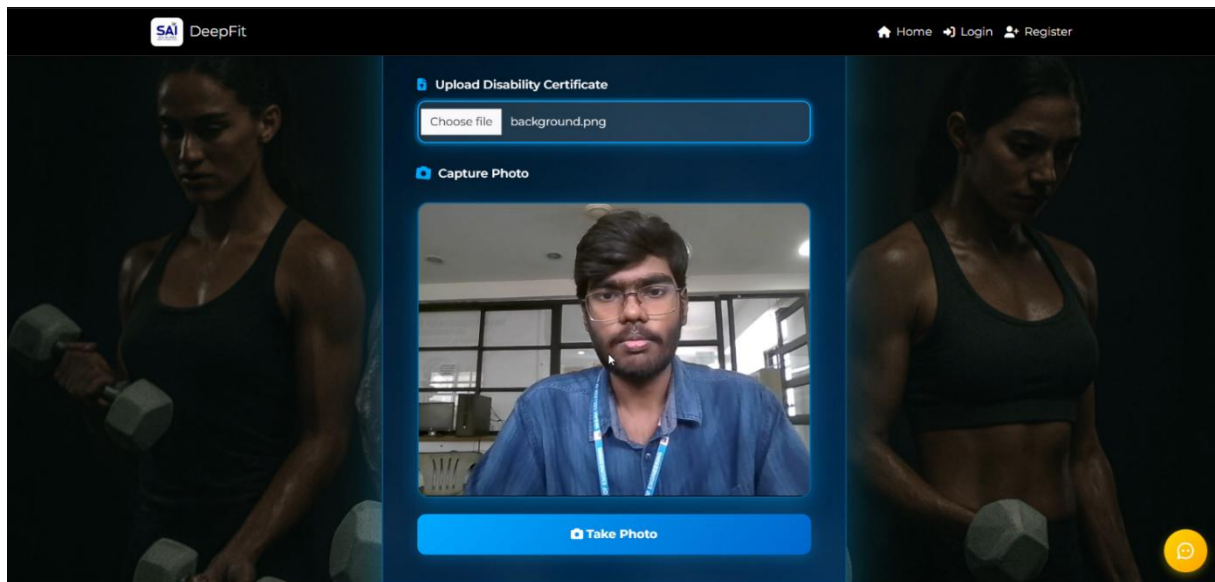
The image shows a web application interface for "Blind Certificate and Photo Capture". The header includes the "SAI DeepFit" logo and navigation links for Home, Login, and Register. The main content area is a dark blue panel with the title "Blind Certificate and Photo Capture" in green. It contains several input fields: "Upload Disability Certificate" with a "Choose file" button and the text "background.png", and "Capture Photo" with a "Take Photo" button. The background of the page features a woman in a black sports bra and leggings, holding a dumbbell.

Fig 4.34: Blind Certificate and Photo Capture

The **Fig 4.34** This interface provides registration verification with certificate upload and camera capture. It ensures identity validation with a user-friendly design.

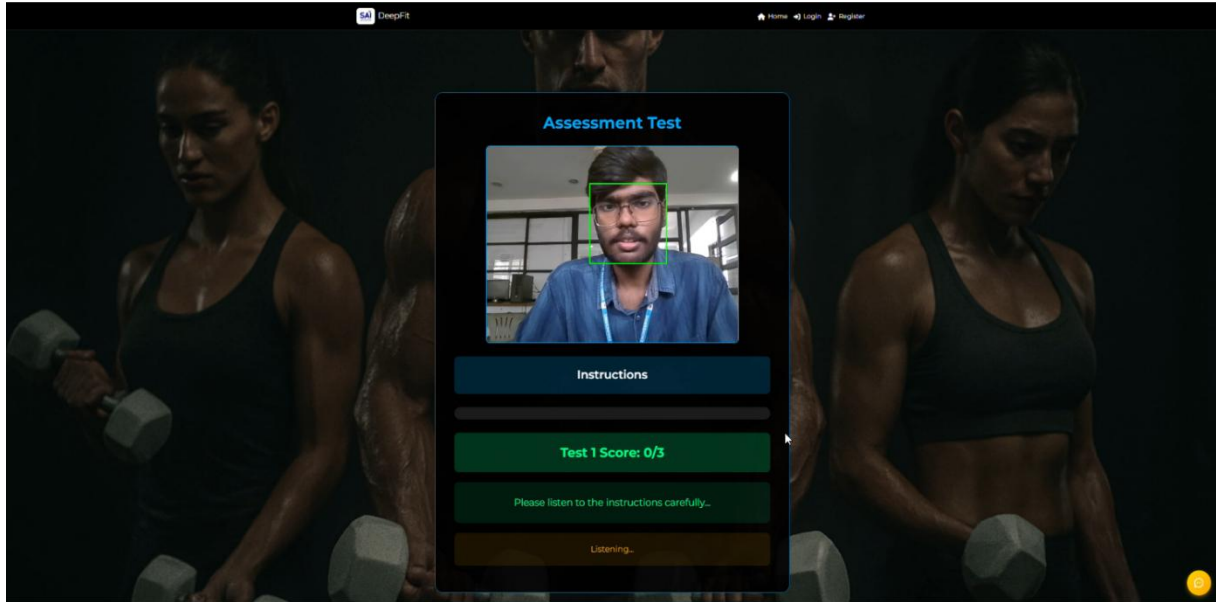


Fig 4.35: Blind Person Facial Detection Page

The **Fig 4.35** screen shows the instruction phase with facial detection before starting the test. It ensures proper positioning and readiness of the user.

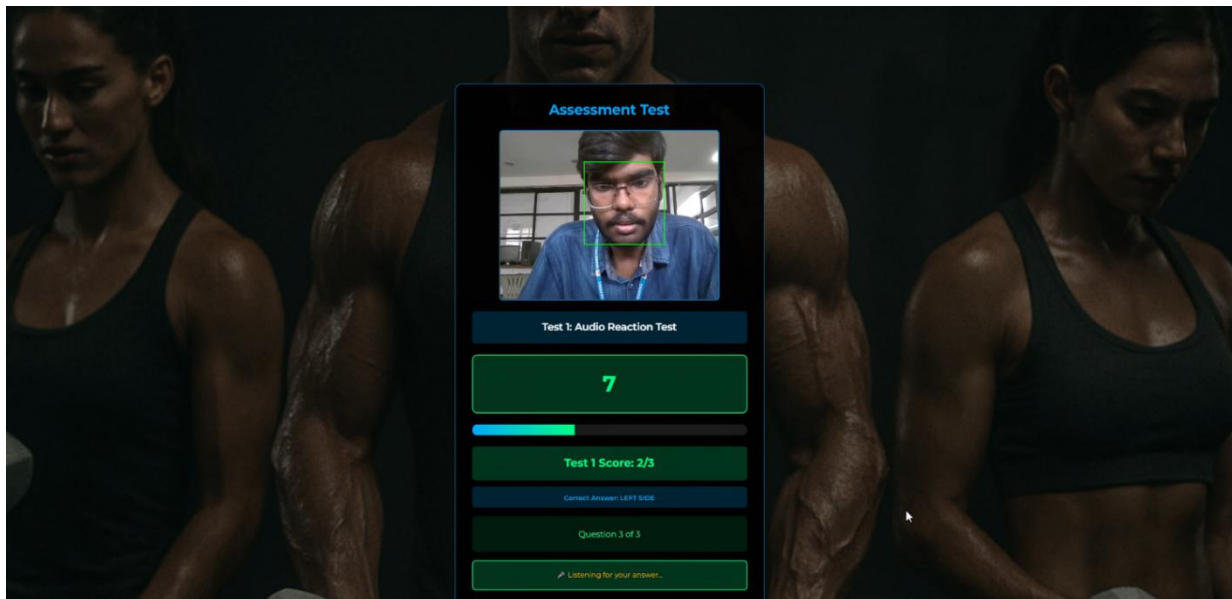


Fig 4.36: Blind Person Assessment Test

The Fig 4.36 This interface displays the audio reaction test with real-time monitoring. It tracks user responses and progress using voice input.

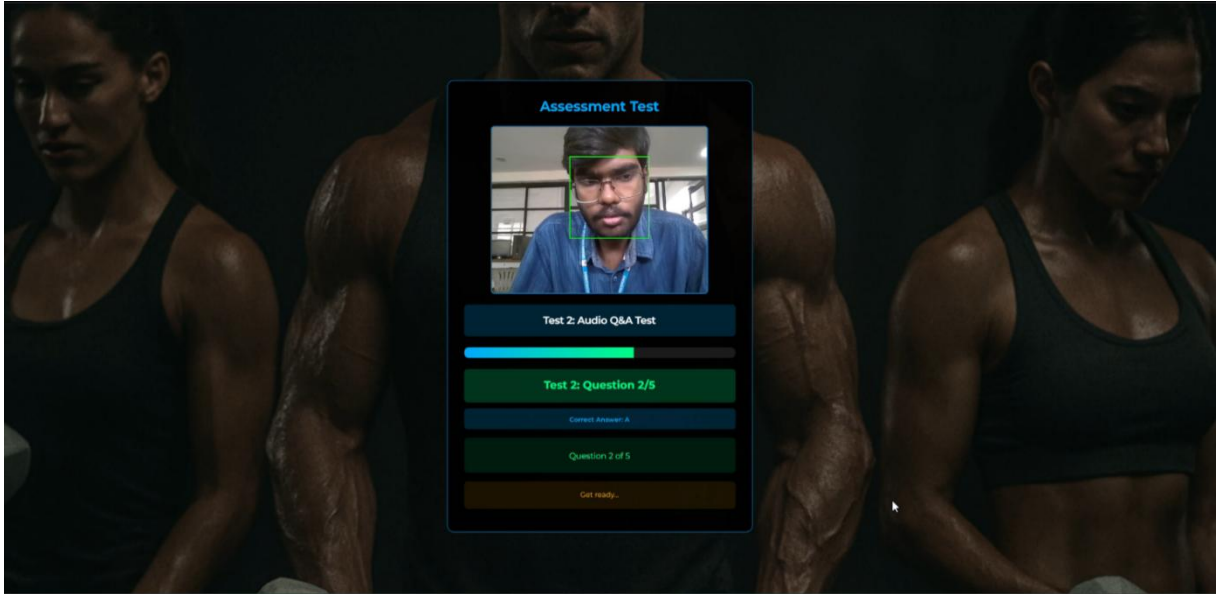


Fig 4.37: Blind Person Assessment Test

The **Fig 4.37** screen shows the audio Q&A test with live camera monitoring. It tracks question progress and evaluates user responses.

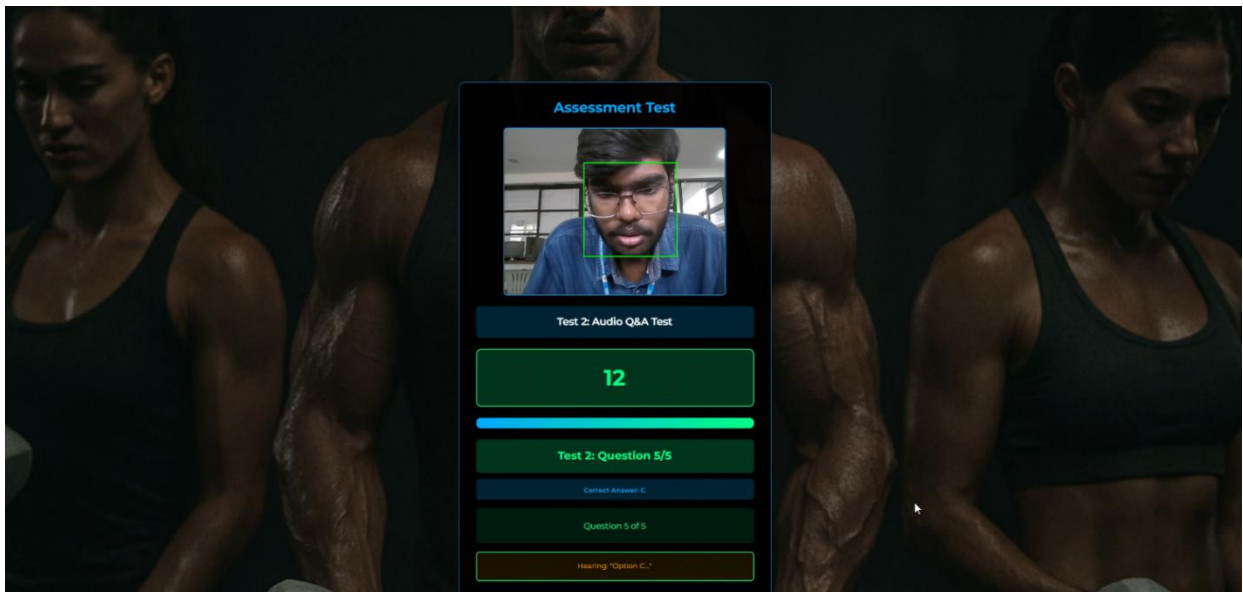


Fig 4.38: Blind Person Assessment Test

The **Fig 4.38** interface displays the blind assessment test with real-time interaction. It monitors user participation using audio and camera input.

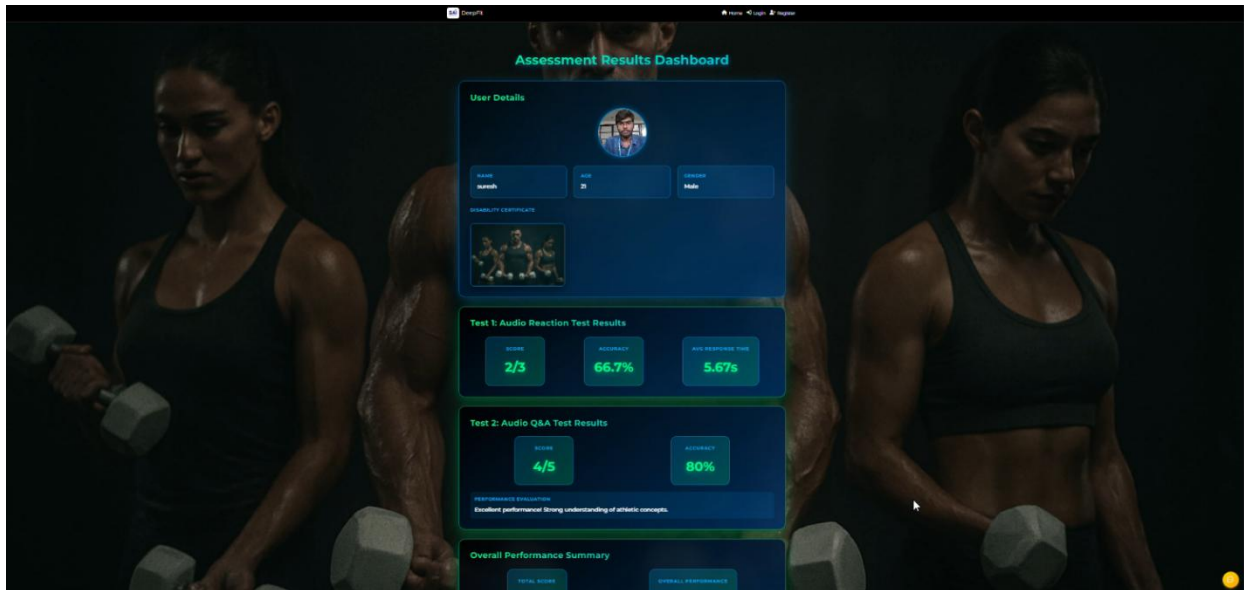


Fig 4.39: Blind Person Assessment Result Page

The **Fig 4.39** dashboard shows detailed assessment results including scores and user information. It provides structured performance analysis.

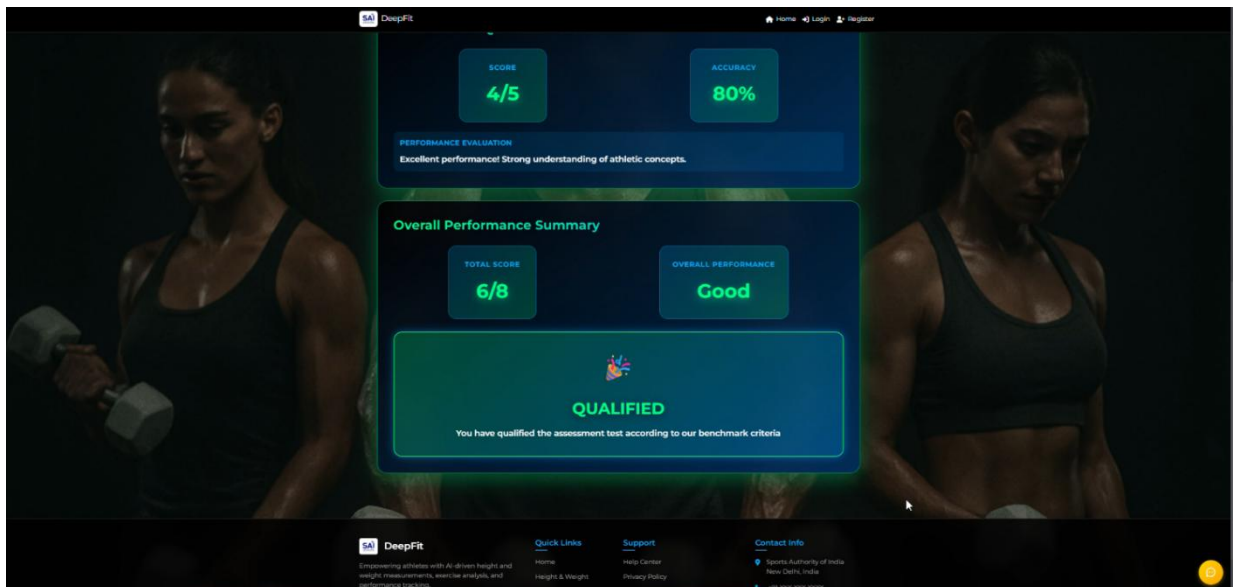


Fig 4.40: Blind Person Result Page

The **Fig 4.40** screen displays the final results with overall score and qualification status. It clearly indicates whether the user is qualified or not.

5. EXPERIMENTAL SETUP & IMPLEMENTATION

5.1 System Specifications

5.1.1 Hardware Requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	Minimum 8 GB
Storage	Minimum 10–20 GB free space
GPU	Optional
Network	Internet access
OS	Windows 10

5.1.2 Software Requirements

Software / Library	Version / Description
Python	3.9 or higher
TensorFlow	Deep learning framework for model training and inference
PyTorch	Used for YOLOv8, MiDaS, and deep learning pipelines
Scikit-learn	Machine learning algorithms (Isolation Forest, One-Class SVM)
SciPy	Scientific computations and optimizations
TorchVision	Image transformations and pretrained models
OpenCV	Real-time video processing and frame extraction
MediaPipe	Pose and facial landmark detection
OpenPose	Human pose estimation
YOLOv8	Real-time object detection
MiDaS	Depth estimation from images/video
ffmpeg-python / PyAV	Video streaming and processing
DeepForensics++	Deepfake detection dataset/tools
pytorchvideo	Video understanding models

PyOD	Outlier detection framework
DeepFace	Face recognition and verification
InsightFace	Advanced face analysis
Flask	Backend API development
flask-cors	Secure cross-origin resource sharing
HTML, CSS, Bootstrap	Frontend UI development
Android Studio (Kotlin)	Mobile application development
Ngrok	Secure tunneling for API access
MongoDB	NoSQL database for storing logs and user data

5.2 Datasets Used in DeepFit AI Project

1. training_data.csv (Weight Estimation Training Dataset)

Location: Root Directory

Size: 350+ records

Purpose:

Used to train the machine learning model (Random Forest Regression) for predicting body weight based on body movement and pose-derived features.

Features:

- avg_rom – Average Range of Motion (in degrees)
- avg_velocity – Average velocity (constant value: 0.1)
- weight_kg – Target variable (5kg, 10kg, 15kg categories)

Description:

This is a synthetic dataset designed to simulate anthropometric measurements derived from pose landmarks. It enables the model to learn patterns between motion characteristics and weight categories.

2. test_dataset.csv (Validation Dataset)

Location: validation_results/

Size: 200 synthetic subjects

Purpose:

Used for validating and testing the accuracy of height and weight estimation models under diverse conditions.

Features:

- subject_id – Unique identifier (SYNTH_000 to SYNTH_199)
- actual_height_cm – Ground truth height
- actual_weight_kg – Ground truth weight
- gender – Male / Female
- age – 18 to 79 years
- body_type – slim, average, athletic, heavy
- ethnicity – african, asian, caucasian, hispanic, mixed
- lighting_condition – good, mixed, poor
- distance_from_camera – 1.5 to 4.0 meters
- pose_quality – excellent, good, fair, poor

Description:

A comprehensive synthetic dataset designed to test system robustness across demographic diversity and environmental variations.

3. Bicep-curl-1 Dataset (Dumbbell Curl Detection)

Location: Bicep-curl-1/

Source: Roboflow Universe

License: CC BY 4.0

Dataset Structure:

- Training Set: 400+ annotated images
- Validation Set: 100+ images
- Test Set: 50+ images

Format: COCO JSON annotations + images

Purpose:

Used to train the YOLOv8 object detection model for identifying and tracking dumbbell curl exercises.

Annotations:

- Bounding boxes
- Pose keypoints (_annotations.coco.json)

Description:

A computer vision dataset with frame-by-frame annotations for accurate exercise detection and correction.

4. jump_data/

- Location: jump_data/
- Purpose: Track vertical jump performance
- Features: Timestamp, Jump_Number, Height_cm
- Description: Session-based performance data

5. AthleteData.csv

- Location: Root Directory
- Purpose: Store athlete information
- Features: Height, Weight, Situps, Jump, Curls
- Description: Links biometric data with performance

6. validation_results/

- Location: validation_results/
- Purpose: Evaluate performance
- Metrics: MAE, RMSE, R^2
- Description: Contains validation and analysis results

7. Pre-trained Models

- MediaPipe Holistic – Pose estimation
- MiDaS Depth Model – Depth estimation
- YOLOv8 – Exercise detection

Code Implementation:

App.py :

```
from flask import Flask, render_template, request, session, redirect, url_for, send_from_directory, jsonify, Response
```

```
from flask_cors import CORS
```

```
from pymongo import MongoClient
```

```

from werkzeug.utils import secure_filename

import os

import bcrypt

import random

import base64

from datetime import datetime, timedelta

from bson import ObjectId

import subprocess

import sys

import time

import secrets

from session_manager import init_session_settings, login_required, get_current_user,
set_user_session

from error_logger import setup_logging, log_error, log_warning, log_info

from dynamic_benchmarks import DynamicBenchmarkSystem

# Initialize Flask app

app = Flask(__name__)

# Dashboard route: protected, fetch user details, render dashboard.html

@app.route('/dashboard')

def dashboard():

    """Cumulative fitness dashboard - requires login to show user profile"""

    if not session or 'user_id' not in session:

        return redirect(url_for('login', next='dashboard'))

    user = None

    try:

        # Direct MongoDB connection

```

```

from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db_conn = client["sih2573"]

print(f'Looking for user with ID: {session['user_id']}') # Debug

user_doc = db_conn.users.find_one({'_id': ObjectId(session['user_id'])})

print(f'Found user doc: {user_doc}') # Debug

if user_doc:

    user = {

        'name': user_doc.get('name', 'N/A'),

        'age': user_doc.get('age', 'N/A'),

        'gender': user_doc.get('gender', 'N/A'),

        'place': user_doc.get('place', 'N/A'),

        'phone': user_doc.get('phone', 'N/A'),

        'email': user_doc.get('email', 'N/A'),

        'profile_pic': user_doc.get('photo', None)

    }

    print(f'Processed user data: {user}') # Debug

else:

    print("No user document found") # Debug

except Exception as e:

    print(f'Dashboard user lookup error: {e}') # Debug

    log_error(f'Dashboard user lookup error: {e}')

# Pass benchmark system status to template

benchmark_status = {

```

```

        'system_available': benchmark_system.athlete_data is not None,

        'total_athletes': len(benchmark_system.athlete_data) if benchmark_system.athlete_data is not
        None else 0

    }

    return render_template('dashboard.html', user=user, benchmark_status=benchmark_status)

@app.route('/user_dashboard')
@login_required
def user_dashboard():
    """User-specific dashboard requiring login"""

    db = get_db()

    if db is None:

        log_error("Dashboard error: Database connection failed.")

        return render_template('dashboard.html', user=None, error='Database connection failed. Please
        contact admin.')

    user_id = session.get('user_id')

    if not user_id:

        return redirect(url_for('login'))

    try:

        user = db.users.find_one({'_id': ObjectId(user_id)})

        if not user:

            return render_template('dashboard.html', user=None, error='User not found in database.')

        # Convert ObjectId to string for template

        user['_id'] = str(user['_id'])

        # Extract required fields, provide defaults if missing

        profile = {

            'name': user.get('name', 'N/A'),

```

```

        'age': user.get('age', 'N/A'),
        'gender': user.get('gender', 'N/A'),
        'place': user.get('place', 'N/A'),
        'phone': user.get('phone', 'N/A'),
        'profile_pic': user.get('photo', user.get('profile_pic', None)), # Prefer 'photo' field, fallback to
'profile_pic'
        '_id': user['_id'],
        'email': user.get('email', 'N/A')
    }

    return render_template('dashboard.html', user=profile, error=None)

except Exception as e:

    log_error(f'Dashboard error: {e}')

    return render_template('dashboard.html', user=None, error=f'Error loading dashboard: {e}')

# Set up logging first
logger = setup_logging('deepfit')

log_info("Starting DeepFit application")

# Configure secure sessions
app.secret_key = secrets.token_hex(32) # Use 32 bytes for extra security

init_session_settings(app)

# Configure CORS for entire application
CORS(app, resources={
    r"/*": {
        "origins": "*",
        "supports_credentials": True,
        "allow_headers": ["Content-Type", "Authorization"],
        "expose_headers": ["Content-Range", "X-Content-Range"]
    }
})

```



```

    }
})

# Helper to obtain a MongoDB database connection used by request handlers.

# This lazily tries to (re)connect if a global `db` isn't available. We import
# `connect_mongodb` locally so the function can be defined before the module
#-level call to `connect_mongodb()` later in the file. The function returns
# the DB object on success or None on failure.

def get_db():
    global client, db, mongodb_connected

    try:
        # If a db object already exists and is truthy, return it immediately
        if 'db' in globals() and db is not None:
            return db

    except NameError:
        pass

    # Attempt to import a connector and establish a connection
    try:
        from db_config import connect_mongodb

        client_conn, db_conn = connect_mongodb()

        if db_conn:
            # Persist in module globals for other code to use
            globals()['client'] = client_conn
            globals()['db'] = db_conn
            globals()['mongodb_connected'] = True

            return db_conn

```

```

except Exception as e:

    try:

        # Use logging helper if available

        log_error(f'get_db error: {e}')

    except Exception:

        print(f'get_db error: {e}')

    return None

# Import and register blueprints after CORS setup

from situp_blueprint import situp_bp

from vertical_jump_blueprint import vertical_jump_bp

from dumbbell_blueprint import dumbbell_bp

from benchmark_routes import benchmark_bp

from chatbot_routes import chatbot_bp

app.register_blueprint(situp_bp, url_prefix='/situp')

app.register_blueprint(vertical_jump_bp, url_prefix='/vertical_jump')

app.register_blueprint(dumbbell_bp, url_prefix='/dumbbell')

app.register_blueprint(benchmark_bp)

app.register_blueprint(chatbot_bp)


# Global error and request handlers

@app.errorhandler(404)

def not_found_error(error):

    log_warning(f'Page not found: {request.url}')

    return render_template('errors/404.html'), 404

@app.errorhandler(500)

```

```

def internal_error(error):

    log_error(error, {'url': request.url, 'method': request.method})

    db = get_db() # Get a fresh database connection

    return render_template('errors/500.html'), 500

@app.errorhandler(Exception)
def handle_exception(error):

    log_error(error, {'url': request.url, 'method': request.method})

    return render_template('errors/500.html'), 500


@app.before_request
def before_request():

    # Log all requests

    log_info(f'{request.method} {request.url}')

    # Ensure database connection is available

    if get_db() is None:

        log_error("Database connection unavailable")

        return render_template('errors/500.html'), 500

    return render_template('errors/500.html'), 500

# Configure session

app.config['SESSION_TYPE'] = 'filesystem'

app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(days=7) # Session expires after 7
days

app.config['SESSION_COOKIE_SECURE'] = True # Only send cookie over HTTPS

app.config['SESSION_COOKIE_HTTPONLY'] = True # Prevent JavaScript access to session cookie

# Global variables for situps detection

camera = None

```

```

is_recording = False

situps_detector = None

exercise_stats = {
    'reps': 0,
    'feedback': 'Get Ready',
    'exercise': 'situps',
    'form_percentage': 0
}

def generate_frames():
    """Generate video frames for streaming"""
    import cv2

    global camera, is_recording, exercise_stats, situps_detector

    while is_recording and camera is not None:
        success, frame = camera.read()

        if not success:
            break

        # Process frame for situps detection
        if situps_detector:
            frame = situps_detector.detect_situps(frame)

            stats = situps_detector.get_stats()

            exercise_stats.update(stats)

        # Encode frame
        ret, buffer = cv2.imencode('.jpg', frame)

```

```

        frame = buffer.tobytes()

        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')

# MongoDB setup
from db_config import connect_mongodb
client, db = connect_mongodb()
mongodb_connected = db is not None
# Initialize dynamic benchmark system
benchmark_system = DynamicBenchmarkSystem(
    mongo_uri="mongodb://localhost:27017/",
    db_name="sih2573"
)
# Set up collection references
if mongodb_connected:
    users = db['users']
    height_videos = db['height_videos']
    exercise_sessions = db['exercise_sessions']
    exercise_results = db['exercise_results']
else:
    users = height_videos = exercise_sessions = exercise_results = None

```

app_situp.py:

```

from flask import Flask, render_template, Response, jsonify
import cv2

```

```

import mediapipe as mp

import numpy as np

app = Flask(__name__)

class SitupsCounter:

    def __init__(self):

        self.mp_pose = mp.solutions.pose

        self.pose = self.mp_pose.Pose(min_detection_confidence=0.5, min_tracking_confidence=0.5)

        self.mp_draw = mp.solutions.drawing_utils

        self.count = 0

        self.stage = None

        self.feedback = "Get Ready"

    def calculate_angle(self, a, b, c):

        a = np.array(a)

        b = np.array(b)

        c = np.array(c)

        radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1], a[0]-b[0])

        angle = np.abs(radians*180.0/np.pi)

        if angle > 180.0:

            angle = 360-angle

        return angle

    def process_frame(self, frame):

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        results = self.pose.process(rgb)

```

```

if results.pose_landmarks:

    landmarks = results.pose_landmarks.landmark

    # Get coordinates

    shoulder = [landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].x,
                landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].y]

    hip = [landmarks[self.mp_pose.PoseLandmark.LEFT_HIP.value].x,
           landmarks[self.mp_pose.PoseLandmark.LEFT_HIP.value].y]

    knee = [landmarks[self.mp_pose.PoseLandmark.LEFT_KNEE.value].x,
            landmarks[self.mp_pose.PoseLandmark.LEFT_KNEE.value].y]

    # Calculate angle

    angle = self.calculate_angle(shoulder, hip, knee)

    # Situp logic

    if angle > 160:

        if self.stage != "down":

            self.stage = "down"

            self.feedback = "Go Up!"

    elif angle < 90 and self.stage == "down":

        self.stage = "up"

        self.count += 1

        self.feedback = f'Rep {self.count}! Go Down'

    # Draw pose

```

```

        self.mp_draw.draw_landmarks(frame, results.pose_landmarks,
self.mp_pose.POSE_CONNECTIONS)

    # Draw info

    cv2.putText(frame, f'Count: {self.count}', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)

    cv2.putText(frame, self.feedback, (10, 70), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255,
0), 2)

    cv2.putText(frame, f'Angle: {int(angle)}', (10, 110), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
(255, 255, 255), 2)

    else:

        self.feedback = "No pose detected"

        cv2.putText(frame, self.feedback, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255),
2)

    return frame

def get_stats(self):
    return {
        'reps': self.count,
        'feedback': self.feedback,
        'form_percentage': min(100, self.count * 10)
    }

def reset(self):
    self.count = 0
    self.stage = None
    self.feedback = "Get Ready"

```



```

# Global variables

camera = None

counter = SitupsCounter()

is_recording = False


def generate_frames():

    global camera, is_recording

    while is_recording and camera:

        success, frame = camera.read()

        if not success:

            break

        frame = counter.process_frame(frame)

        ret, buffer = cv2.imencode('.jpg', frame)

        frame = buffer.tobytes()

        yield (b'--frame\r\n'

               b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')


@app.route('/')

def index():

    """Main page"""

    return render_template('index_situp.html')


@app.route('/start_camera')

def start_camera():

```

```

global camera, is_recording

try:

    camera = cv2.VideoCapture(0)

    if not camera.isOpened():

        return jsonify({'status': 'error', 'message': 'Camera not found'})

    is_recording = True

    return jsonify({'status': 'success', 'message': 'Camera started'})

except Exception as e:

    return jsonify({'status': 'error', 'message': str(e)})


@app.route('/stop_camera')
def stop_camera():

    global camera, is_recording

    is_recording = False

    if camera:

        camera.release()

        camera = None

    return jsonify({'status': 'success', 'message': 'Camera stopped'})


@app.route('/reset_counter')
def reset_counter():

    counter.reset()

    return jsonify({'status': 'success', 'message': 'Counter reset'})


@app.route('/video_feed')

```

```
def video_feed():  
    """Video streaming route"""  
    return Response(generate_frames(),  
                    mimetype='multipart/x-mixed-replace; boundary=frame')  
  
@app.route('/get_stats')  
def get_stats():  
    return jsonify(counter.get_stats())  
  
if __name__ == '__main__':  
    app.run(debug=True, host='127.0.0.1', port=5000)
```

5.3 Methodology / Algorithm

Stage 1: Video Input & Streaming

- Video is captured via webcam, uploaded file, or mobile app
- Streaming handled using OpenCV and ffmpeg

Stage 2: Frame Processing

- Video is divided into frames
- Preprocessing applied (resize, normalization)

Stage 3: Object & Pose Detection

- YOLOv8 detects objects (person, phone, etc.)
- MediaPipe / OpenPose tracks body movements and posture

Stage 4: Depth & Face Analysis

- MiDaS estimates depth to detect abnormal positioning
- DeepFace / InsightFace performs face recognition and verification

Stage 5: Fraud / Cheat Detection

- Behavioral anomalies detected using:
 - Isolation Forest
 - One-Class SVM
 - PyOD models
- Deepfake detection using DeepForensics++ and video models

Stage 6: Backend Processing

- Flask APIs handle communication between frontend and models
- flask-cors ensures secure API access

Stage 7: Data Storage & Security

- Logs and results stored in MongoDB
- Blockchain (via web3.py) ensures tamper-proof records
- Hashing (SHA256/MD5) used for data integrity

Stage 8: Output & Alerts

- Results displayed on web dashboard (HTML, Bootstrap)
- Alerts generated for suspicious activities
- Mobile app receives real-time updates via API

6. RESULTS

The system successfully performs accurate exercise analysis using AI-based pose detection and provides real-time performance evaluation. DeepFit AI ensures fair sports talent identification for academies and large-scale talent discovery programs. The visual dashboard enables users to track progress easily, and the use of a single camera ensures accessibility and cost-effectiveness. The system also maintains consistency in evaluation, reducing dependency on manual supervision. Overall, it delivers reliable and unbiased performance results.

The implementation of DeepFit AI demonstrates significant improvement in fitness assessment by introducing automation, consistency, and standardized evaluation methods. The system was evaluated on multiple exercises such as Sit-ups, Vertical Jump, Sit-and-Reach, and Dumbbell Curls, where it accurately analyzed body movements using pose estimation models. Features like repetition counting, joint tracking, and motion analysis produced consistent and dependable results, minimizing errors commonly associated with manual assessment. Additionally, the system provides instant feedback along with graphical insights, enabling users to better understand their performance and areas for improvement.

The platform also proved effective in ensuring inclusivity and reliability. The disabled assessment module allowed **blind** users to perform audio-based tests and **deaf** users to participate through visual and movement-based evaluations, confirming accessibility without compromising accuracy. Furthermore, the integration of cheat detection mechanisms ensured that only authentic inputs were considered, improving trust in the evaluation process. The inclusion of an AI chatbot enhanced user experience by offering real-time guidance and support. Overall, the system demonstrates that DeepFit AI is a scalable, accurate, and unbiased solution for modern fitness assessment and talent identification.

7. CONCLUSION

DeepFit AI eliminates **human bias** and **evaluation bias** by providing an automated and standardized fitness assessment system. It ensures fair, accurate, and scalable evaluation while improving accessibility for athletes in rural and remote areas. The system supports data-driven **decision-making** for talent identification. It also simplifies the assessment process by removing the need for **specialized equipment**. Overall, it represents a modern and efficient solution for sports performance evaluation.

The inclusion of modules for blind and deaf users highlights the system's focus on inclusivity. By providing audio-based and visual-based assessments, the system ensures equal opportunities for all individuals. The integration of an AI chatbot and cheat detection mechanisms further improves usability and reliability, making the system more robust.

The proposed system enhances the overall efficiency of sports performance assessment through automation, intelligent analysis, and real-time feedback. By combining artificial intelligence with computer vision techniques, it provides accurate monitoring of physical activities while reducing manual intervention and operational complexity. The platform offers a user-friendly environment for coaches, trainers, and participants to track progress, analyse results, and improve performance effectively. Its scalable and flexible architecture also allows future integration of advanced analytics, and support for multiple sports activities, making it a reliable and future-oriented solution in modern sports technology.

FUTURE SCOPE

In the future, the system can be enhanced by adding more fitness tests and improving the accuracy of AI models through better training datasets. Features such as personalized feedback, real-time coaching, and advanced analytics can be introduced to provide deeper insights into user performance.

The platform can also be scaled for large-scale deployment in sports academies and national-level talent identification programs. With further improvements, DeepFit AI has the potential to become a standard solution for fair, efficient, and inclusive fitness assessment systems.

8. REFERENCES

- [1] MediaPipe Documentation : <https://developers.google.com/mediapipe>
- [2] Google AI Studio : <https://aistudio.google.com>
- [3] OpenCV Library : Open Source Computer Vision Library : <https://opencv.org>
- [4] Redmon, Joseph, Farhadi, Ali (2018). YOLOv3: An Incremental Improvement.
- [5] PyTorch (Meta AI). Deep learning framework for computer vision: <https://pytorch.org>
- [6] Sports Authority of India (SAI). Talent Identification and Development Guidelines : <https://sportsauthorityofindia.nic.in>
- [7] Machine Learning in Sports Talent Identification :
https://www.researchgate.net/publication/362910188_Machine_Learning_in_Sports_Talent_Identification_A_Systematic_Review
- [8] Accuracy Evaluation of 3D Pose Estimation with MediaPipe Pose for Physical Exercises :
https://www.researchgate.net/publication/374081734_Accuracy_Evaluation_of_3D_Pose_Estimation_with_MediaPipe_Pose_for_Physical_Exercises

PROTOTYPE LINKS :

GitHub : <https://github.com/Arunchand19/DeepFitAI>

Prototype Demo Video (Complete Detection Analysis) :

<https://drive.google.com/file/d/1VmyARNuvZFTq1fVrJ8BpgBlkD0CoAdC3/view>

Blind and Deaf Demo Video :

https://drive.google.com/file/d/1E3w63pGLb2rTuG5ARDwa-2_YnpMzsJ7-/view?usp=sharing

Mobile App Emulator Demo Video :

<https://drive.google.com/file/d/1Vi5UHbHT5Mwwqn0hTZH4K6LI0GHXjbMQ/view?usp=sharing>

DeepFit Website Deploy Link :

<https://afridpasha1983-deepfit-ai.hf.space/>